DATA ENGINEERING

INTERVIEW PREP BOOKLET

300+ Real Interview Questions with Answers & Code

Netflix · IBM · S&P Global · Teradata · Databricks · Snowflake · Stripe · Airbnb · Meta · Amazon

SQL · PySpark · Kafka · Airflow · dbt · Delta Lake · System Design · Behavioral

CHAPTER 1 — SQL FUNDAMENTALS (Questions 1–60)

1.1 Window Functions

Q1 [Netflix] What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?

All three rank rows within a partition but differ on ties:

- ROW_NUMBER(): always unique integers (1,2,3,4). No ties ever. Use for deduplication / top-1 per group.
- RANK(): ties share rank; next rank skips (1,1,3). Use for leaderboards where gaps matter.
- DENSE_RANK(): ties share rank; next rank is consecutive (1,1,2). Use for relative standings without gaps.

```
-- Deduplication (ROW_NUMBER):
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY updated_at DESC) rn
  FROM users) t WHERE rn = 1;
```

Q2 [Airbnb] Write a query to compute the 7-day rolling average revenue per host.

```
SELECT host_id, booking_date,
  AVG(revenue) OVER (
    PARTITION BY host_id ORDER BY booking_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS rolling_7d_avg
FROM host_bookings;
```

ROWS BETWEEN uses physical row count (correct for dates). RANGE BETWEEN uses value range (careful with duplicate dates).

Q3 [Stripe] Calculate month-over-month revenue growth % using window functions.

```
WITH monthly AS (
  SELECT DATE_TRUNC('month',txn_date) mo, SUM(amount) rev FROM txns GROUP BY 1
)
SELECT mo, rev,
  LAG(rev) OVER (ORDER BY mo) prev_rev,
  ROUND(100.0*(rev - LAG(rev) OVER (ORDER BY mo)) / NULLIF(LAG(rev) OVER (ORDER BY
mo),0),2) mom_pct
FROM monthly;
```

Q4 [IBM] What is the difference between LEAD() and LAG()? Give a real scenario.

LAG() looks back N rows; LEAD() looks forward N rows — both without a self-join.

```
-- Days between consecutive transactions per account
SELECT account_id, txn_date,
  DATEDIFF(txn_date, LAG(txn_date) OVER (PARTITION BY account_id ORDER BY txn_date))
gap_days
FROM transactions;
```

Q5 [Meta] Find the top 3 products by sales within each category.

```
SELECT * FROM (
  SELECT category, product, sales,
    DENSE_RANK() OVER (PARTITION BY category ORDER BY sales DESC) dr
  FROM product_sales
) t WHERE dr <= 3;
```

Q6 [Databricks] Compute a running total of orders per customer, reset each year.

```
SELECT customer_id, order_date,
  SUM(amount) OVER (
    PARTITION BY customer_id, YEAR(order_date)
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS ytd_total
FROM orders;
```

Q7 [Amazon] What does NTILE(4) do? Write a query to bucket customers into revenue quartiles.

NTILE(n) divides rows into n equal-sized buckets and assigns a bucket number (1..n).

```
SELECT customer_id, total_spend,
  NTILE(4) OVER (ORDER BY total_spend DESC) AS quartile
FROM customer_spend;
```
-- Q1 = top 25%, Q4 = bottom 25%

Q8 [Snowflake] Explain FIRST_VALUE() and LAST_VALUE(). Why does LAST_VALUE() often return unexpected results?

FIRST_VALUE() returns the first value in the window frame. LAST_VALUE() returns the last value of the frame.

Default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW — so LAST_VALUE() sees only up to the current row, not the end of the partition.

```
-- Fix: explicitly extend the frame
LAST_VALUE(col) OVER (PARTITION BY x ORDER BY y
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

Q9 [S&P Global] Write a query to find the second-highest salary in each department, handling ties.

```
SELECT dept, salary FROM (
  SELECT dept, salary,
    DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) dr
  FROM employees
) t WHERE dr = 2;
```

Q10 [Netflix] Calculate the percentage of total revenue each content category contributes.

```
SELECT category,
  SUM(revenue) AS cat_rev,
  ROUND(100.0 * SUM(revenue) / SUM(SUM(revenue)) OVER (), 2) AS pct_of_total
FROM content_revenue
GROUP BY category;
```

SUM(SUM(revenue)) OVER () is a window over the entire result set — no PARTITION BY.

1.2 JOINs & Set Operations

Q11 [General] Explain every type of JOIN and when you would use each.

- INNER JOIN: only matching rows from both sides.
- LEFT JOIN: all left rows; NULL on right where no match.
- RIGHT JOIN: all right rows (flip table order instead).
- FULL OUTER JOIN: all rows from both; NULLs where no match.
- CROSS JOIN: Cartesian product — every row × every row. Use to generate combinations.
- SELF JOIN: table joined to itself (org hierarchy, adjacency lists).
- SEMI JOIN (EXISTS/IN): return left rows that have at least one match on right.
- ANTI JOIN (NOT EXISTS / LEFT JOIN WHERE NULL): left rows with NO match on right.

Q12 [S&P Global] Find all instruments in Table A that are NOT in Table B. Which approach is safest?

```
-- SAFEST: LEFT JOIN anti-join
SELECT a.instrument_id FROM instruments_a a
LEFT JOIN instruments_b b ON a.instrument_id = b.instrument_id
WHERE b.instrument_id IS NULL;
```

-- NOT IN is DANGEROUS: if instruments_b has even one NULL, the entire result is empty.

-- NOT EXISTS is safe and stops at first match (efficient).

Q13 [IBM] What causes a JOIN to produce duplicate rows (fan-out)? How do you detect and fix it?

Fan-out occurs when the join key is not unique on one side (many-to-one becomes many-to-many).

Detect: COUNT(*) before join vs after. If count grows unexpectedly, you have a fan-out.

```
-- Fix: deduplicate before joining
WITH deduped AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY promo_id ORDER BY updated_at DESC) rn
  FROM promotions
)
SELECT o.*, d.promo_name FROM orders o
```

```
JOIN deduped d ON o.promo_id = d.promo_id AND d.rn = 1;
```

Q14 [Meta] What is the difference between UNION and UNION ALL?

UNION removes duplicate rows (implicit DISTINCT — expensive sort/hash). UNION ALL keeps all rows including duplicates and is much faster.

Use UNION only when you actually need deduplication. In ETL pipelines, UNION ALL is almost always preferred.

Q15 [Amazon] Write a self-join to find all pairs of employees in the same department.

```
SELECT a.name e1, b.name e2, a.dept
FROM employees a
JOIN employees b ON a.dept = b.dept AND a.emp_id < b.emp_id;
```

-- a.emp_id < b.emp_id avoids (A,B) and (B,A) duplicates and (A,A) self-pairs.

Q16 [Databricks] When would you use a CROSS JOIN in a data warehouse? Give a practical example.

Use CROSS JOIN to generate a dense grid of all combinations, e.g. all date × product combos for a sales matrix with zeros for missing data.

```
SELECT d.date, p.product_id, COALESCE(s.revenue,0) revenue
FROM date_dim d CROSS JOIN products p
LEFT JOIN sales s ON s.date=d.date AND s.product_id=p.product_id;
```

Q17 [Teradata] Explain the difference between INTERSECT and EXCEPT (MINUS).

INTERSECT: rows present in BOTH queries (like INNER JOIN without duplication).

EXCEPT / MINUS: rows in first query NOT in second (like anti-join).

Both remove duplicates by default. Add ALL suffix (EXCEPT ALL) to keep duplicates.

1.3 Aggregation & GROUP BY

Q18 [Stripe] Find users who made purchases in at least 3 different months.

```
SELECT user_id FROM (
  SELECT user_id, DATE_TRUNC('month',txn_date) mo FROM transactions GROUP BY 1,2
) t
GROUP BY user_id HAVING COUNT(DISTINCT mo) >= 3;
```

Q19 [Netflix] What is the difference between WHERE and HAVING?

WHERE filters rows BEFORE aggregation (cannot reference aggregate functions).

HAVING filters groups AFTER aggregation (can reference aggregate functions).

```
SELECT dept, AVG(salary) avg_sal FROM employees
WHERE status = 'active'            -- pre-aggregation filter
GROUP BY dept
HAVING AVG(salary) > 80000;         -- post-aggregation filter
```

Q20 [IBM] Explain GROUPING SETS, ROLLUP, and CUBE with an example.

```
-- ROLLUP: hierarchical subtotals
SELECT region, country, SUM(sales) FROM sales GROUP BY ROLLUP(region, country);
-- Generates: (region,country), (region), (grand total)

-- CUBE: all possible combinations
SELECT region, product, SUM(sales) FROM sales GROUP BY CUBE(region, product);

-- GROUPING SETS: explicit control
SELECT region, product, SUM(sales) FROM sales
GROUP BY GROUPING SETS ((region,product),(region),(product),());
```

Q21 [Amazon] Find the department with the highest average salary, excluding departments with fewer than 5 employees.

```
SELECT dept, AVG(salary) avg_sal
FROM employees
GROUP BY dept
HAVING COUNT(*) >= 5
```

```
ORDER BY avg_sal DESC
LIMIT 1;
```

Q22 [Snowflake] What does COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col) return differently?

- COUNT(*): counts all rows including NULLs.
- COUNT(col): counts non-NULL values of col.
- COUNT(DISTINCT col): counts unique non-NULL values.

```
SELECT COUNT(*), COUNT(email), COUNT(DISTINCT email) FROM users;
```

-- Results differ when email has NULLs or duplicates.

Q23 [S&P Global] Write a query to pivot rows into columns (e.g., quarterly sales by year).

```
-- CASE-based pivot (universal)
SELECT year,
  SUM(CASE WHEN quarter='Q1' THEN revenue END) Q1,
  SUM(CASE WHEN quarter='Q2' THEN revenue END) Q2,
  SUM(CASE WHEN quarter='Q3' THEN revenue END) Q3,
  SUM(CASE WHEN quarter='Q4' THEN revenue END) Q4
FROM quarterly_sales GROUP BY year;
```

-- Snowflake/SQL Server also have native PIVOT syntax.

Q24 [Teradata] How does Teradata handle GROUP BY with SET vs MULTISET tables?

In SET tables Teradata enforces row uniqueness at write time (expensive duplication check).

MULTISET allows duplicates. GROUP BY behavior is the same; difference is in how rows are stored.

For large aggregation loads, MULTISET + GROUP BY in the SELECT is far faster than SET table deduplication.

1.4 NULL Handling

Q25 [General] List 5 NULL gotchas that cause bugs in SQL queries.

1. NULL = NULL is UNKNOWN, not TRUE. Use IS NULL / IS NOT NULL.
2. NOT IN with NULLs: if subquery returns any NULL, entire NOT IN returns empty set.
3. Aggregate functions ignore NULLs — AVG(col) divides by count of non-NULLs only.
4. NULL arithmetic: 5 + NULL = NULL. Propagates through all operations.
5. != comparison: rows where col IS NULL are NOT returned by col != 'value'.

```
-- Safe: include NULLs explicitly
SELECT * FROM emp WHERE salary != 50000 OR salary IS NULL;
```

Q26 [IBM] What is the difference between COALESCE(), NULLIF(), and NVL()?

- COALESCE(a,b,c): returns first non-NULL value. Standard SQL, all databases.
- NULLIF(a,b): returns NULL if a=b, else returns a. Use to avoid divide-by-zero.
- NVL(a,b): Teradata/Oracle equivalent of COALESCE with 2 args.

```
SELECT COALESCE(phone, email, 'unknown') contact FROM users;
SELECT revenue / NULLIF(units,0) avg_price FROM sales;  -- safe division
```

Q27 [Snowflake] How do you handle NULLs in ORDER BY? Is NULL first or last?

SQL standard: NULLs are "larger than all values" so they appear LAST in ASC, FIRST in DESC.

PostgreSQL default: NULLs last in ASC. MySQL: NULLs first in ASC.

```
-- Control NULL sort position explicitly
ORDER BY salary ASC NULLS LAST;    -- push NULLs to end
ORDER BY salary DESC NULLS FIRST;  -- standard DESC behavior
```

1.5 CTEs, Subqueries & Recursion

Q28 [General] CTE vs Subquery vs Temp Table — when do you use each?

• CTE (WITH clause): readable, reusable within query, no physical materialization (usually). Use for complex multi-step logic.

• Subquery: inline, good for one-off scalar lookups. Gets unreadable when nested deeply.

• Temp Table: materializes data physically. Use when: CTE is referenced many times, planner makes bad estimates, or you need an index on intermediate data.

Q29 [IBM] Write a recursive CTE to traverse an employee-manager hierarchy.

```
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 0 AS depth
  FROM employees WHERE manager_id IS NULL   -- anchor: CEO
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.depth+1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org ORDER BY depth, name;
```

Always add WHERE depth < 10 to prevent infinite loops from bad data.

Q30 [Netflix] Find users who watched content every single day in January 2024.

```
SELECT user_id FROM watch_events
WHERE watch_date BETWEEN '2024-01-01' AND '2024-01-31'
GROUP BY user_id
HAVING COUNT(DISTINCT watch_date) = 31;
```

Q31 [Stripe] Find customers who placed an order in 2023 but NOT in 2024.

```
SELECT DISTINCT customer_id FROM orders WHERE YEAR(order_date)=2023
EXCEPT
SELECT DISTINCT customer_id FROM orders WHERE YEAR(order_date)=2024;
```

-- Or LEFT JOIN anti-join if EXCEPT not supported.

Q32 [Amazon] Write a query to find the Nth highest salary without using LIMIT/TOP.

```
-- N=3 example using DENSE_RANK
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) dr FROM employees
) t WHERE dr = 3;
```

Q33 [Meta] What is a lateral join / CROSS APPLY? When is it useful?

LATERAL (PostgreSQL) / CROSS APPLY (SQL Server/Teradata) allows the right side to reference columns from the left side — like a correlated subquery that can return multiple rows.

```
-- Get top 2 orders per customer (without window function)
SELECT c.customer_id, o.order_id, o.amount
FROM customers c
CROSS APPLY (
  SELECT TOP 2 * FROM orders o WHERE o.customer_id = c.customer_id
  ORDER BY o.amount DESC
) o;
```

1.6 Query Optimization & Indexes

Q34 [IBM] A query went from 2s to 20min overnight. Walk me through debugging it.

1. Run EXPLAIN ANALYZE — check for full table scans, bad row estimates, nested loops on large sets.
2. Check statistics freshness — run ANALYZE / COLLECT STATISTICS.
3. Check data volume — did upstream table grow unexpectedly?
4. Check for locks or resource contention.
5. Look at index health — index dropped, fragmented, or invalidated?
6. Check for parameter sniffing (SQL Server) — cached plan built for atypical parameters.
7. Check recent code/schema changes — view or table structure modified?

```
EXPLAIN (ANALYZE, BUFFERS) SELECT ... ;   -- PostgreSQL
EXPLAIN SELECT ... ;                      -- Teradata
```

Q35 [General] Explain B-Tree, Bitmap, and Hash indexes. When does each win?

• B-Tree: balanced tree. Best for range queries (BETWEEN, >, <), ORDER BY, equality. Default in OLTP.

• Bitmap: a bitmap per distinct value. Best for low-cardinality columns in DW (status, region). Terrible for OLTP (row-level lock on bitmap updates).

• Hash: O(1) exact equality only. Cannot range scan. Used internally by DBMS for hash joins.

Q36 [Snowflake] What is a covering index and why does it improve performance?

A covering index includes all columns the query needs, so the engine satisfies the query from the index alone — no heap/table access (no "bookmark lookup").

```
-- Query: SELECT customer_id, amount FROM orders WHERE customer_id = 123
CREATE INDEX idx_cover ON orders(customer_id) INCLUDE (amount);
```

-- Index stores customer_id (key) + amount (included). No row lookup needed.

Q37 [Teradata] What is the leftmost prefix rule for composite indexes?

A composite index on (A, B, C) is usable when filters include A, or A+B, or A+B+C — but NOT B alone or C alone.

```
CREATE INDEX idx ON orders(customer_id, order_date, status);
-- Uses index: WHERE customer_id = 5
-- Uses index: WHERE customer_id = 5 AND order_date > '2024-01-01'
-- Does NOT use: WHERE order_date > '2024-01-01'  (skip leftmost col)
```

Q38 [General] What is partition pruning? Give an example of a query that breaks it.

Partition pruning = optimizer skips partitions that cannot match the filter. Requires filtering directly on the partition column.

```
-- Table partitioned by event_date
-- PRUNES (reads only Jan partition):
SELECT * FROM events WHERE event_date = '2024-01-15';

-- BREAKS pruning (function wraps partition column):
SELECT * FROM events WHERE YEAR(event_date) = 2024;  -- full scan!
-- Fix:
SELECT * FROM events WHERE event_date >= '2024-01-01' AND event_date < '2025-01-01';
```

Q39 [Amazon] What is a correlated subquery? Rewrite one as a JOIN.

Correlated subquery references outer query columns — re-executed for every outer row (slow).

```
-- BAD: correlated subquery (runs N times)
SELECT * FROM orders o
WHERE amount > (SELECT AVG(amount) FROM orders o2 WHERE o2.region = o.region);

-- GOOD: JOIN (runs once)
SELECT o.* FROM orders o
JOIN (SELECT region, AVG(amount) avg_a FROM orders GROUP BY region) r
ON o.region = r.region WHERE o.amount > r.avg_a;
```

Q40 [IBM] What is data skew in SQL/DW? How do you detect and resolve it?

Skew: data unevenly distributed across partitions/nodes. One node does most of the work.

```
-- Detect: which join key values are hot?
SELECT customer_id, COUNT(*) FROM orders GROUP BY 1 ORDER BY 2 DESC LIMIT 10;
```

Fix options: use a surrogate/salted key for joins, pre-aggregate hot keys, use broadcast join for small dims.

1.7 Data Warehouse SQL Patterns

Q41 [S&P Global] Write SQL to implement SCD Type 2 upsert (expire old, insert new).

```
-- Step 1: Expire changed rows
UPDATE dim_customer SET eff_end = CURRENT_DATE-1, is_current=FALSE
WHERE customer_id IN (
  SELECT s.customer_id FROM staging s
  JOIN dim_customer d ON s.customer_id=d.customer_id AND d.is_current=TRUE
  WHERE s.email != d.email OR s.city != d.city
);
```

```
-- Step 2: Insert new versions
INSERT INTO dim_customer (customer_id,email,city,eff_start,eff_end,is_current)
SELECT s.customer_id,s.email,s.city,CURRENT_DATE,'9999-12-31',TRUE
FROM staging s
LEFT JOIN dim_customer d ON s.customer_id=d.customer_id AND d.is_current=TRUE
WHERE d.customer_id IS NULL OR s.email!=d.email OR s.city!=d.city;
```

Q42 [Teradata] What is SCD Type 6 (Hybrid)? When would you use it?

SCD Type 6 = Types 1+2+3 combined. Stores: full history via Type 2 rows, current value overwritten via Type 1 on all rows (current_city), previous value in a separate column via Type 3.

Use when: analysts need current state fast (Type 1 override) AND historical analysis (Type 2 rows) AND quick prior-value lookup (Type 3 column).

Q43 [IBM] Write SQL for a Type 1 SCD (overwrite) merge.

```
MERGE INTO dim_customer AS target
USING staging_customer AS source ON target.customer_id = source.customer_id
WHEN MATCHED THEN UPDATE SET target.email=source.email, target.city=source.city
WHEN NOT MATCHED THEN INSERT (customer_id,email,city) VALUES
(source.customer_id,source.email,source.city);
```

Q44 [Snowflake] Write a query to detect duplicate records and report which columns differ.

```
SELECT id, COUNT(*) cnt FROM orders GROUP BY id HAVING COUNT(*) > 1;

-- Show differing columns between duplicates
SELECT a.order_id, a.amount a_amount, b.amount b_amount
FROM orders a JOIN orders b ON a.order_id=b.order_id AND a.row_key < b.row_key
WHERE a.amount != b.amount;
```

Q45 [Netflix] Write a query to find users who churned (no activity in last 30 days vs prior 30 days).

```
WITH active_prior AS (
  SELECT DISTINCT user_id FROM events
  WHERE event_date BETWEEN CURRENT_DATE-60 AND CURRENT_DATE-31
),
active_recent AS (
  SELECT DISTINCT user_id FROM events
  WHERE event_date >= CURRENT_DATE-30
)
SELECT p.user_id FROM active_prior p
LEFT JOIN active_recent r ON p.user_id=r.user_id
WHERE r.user_id IS NULL;   -- was active, now gone
```

Q46 [Meta] Write SQL to find the median salary per department.

```
-- PERCENTILE_CONT is standard SQL:2003
SELECT dept,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees GROUP BY dept;
```

-- If not available, use NTILE(2) or ROW_NUMBER approach.

Q47 [Amazon] What is the difference between a fact table and a dimension table?

- Fact table: stores measurable, quantitative events (orders, clicks, transactions). Contains foreign keys to dims + additive metrics (amount, quantity, duration).
- Dimension table: stores descriptive context (who, what, where, when). Contains surrogate key + attributes.
- Fact tables are wide and tall; dim tables are wide but shorter.
- Fact metrics can be additive (SUM), semi-additive (AVG balance), or non-additive (ratios).

Q48 [IBM] What is a degenerate dimension? Give an example.

A degenerate dimension is a dimension attribute stored directly in the fact table with no separate dim table — because it has no useful descriptive attributes of its own.

Example: order_number in a fact_order_lines table. It identifies the order but there's nothing meaningful to store in a dim_order. So it lives as a column in the fact table.

Q49 [Teradata] Explain the grain of a fact table and why choosing the wrong grain is catastrophic.

Grain = the most atomic level of measurement each row represents.

Wrong grain example: mixing order-level and order-line-level rows in the same fact table. Aggregations become meaningless — revenue double-counts if you accidentally sum order-level + line-level rows.

Rule: one grain per fact table. State it explicitly in documentation: "Each row represents one order line item."

CHAPTER 2 — ADVANCED SQL & ANALYTICS (Questions 61–90)

Q50 [Netflix] Write a query to calculate cohort retention: % of users from each signup month still active 30/60/90 days later.

```
WITH cohorts AS (
  SELECT user_id, DATE_TRUNC('month',signup_date) cohort_month FROM users
),
activity AS (
  SELECT DISTINCT user_id, DATE_TRUNC('month',event_date) active_month FROM events
)
SELECT c.cohort_month,
  COUNT(DISTINCT c.user_id) cohort_size,
  COUNT(DISTINCT CASE WHEN DATEDIFF(a.active_month,c.cohort_month)=1 THEN a.user_id END)
m1_retained,
  COUNT(DISTINCT CASE WHEN DATEDIFF(a.active_month,c.cohort_month)=2 THEN a.user_id END)
m2_retained
FROM cohorts c LEFT JOIN activity a ON c.user_id=a.user_id
GROUP BY c.cohort_month;
```

Q51 [Stripe] Detect potential fraudulent transactions: same card, amount within $1, within 5 minutes.

```
SELECT a.txn_id, b.txn_id AS dup_txn
FROM transactions a
JOIN transactions b
  ON a.card_id = b.card_id
  AND a.txn_id < b.txn_id
  AND ABS(a.amount - b.amount) < 1
  AND ABS(DATEDIFF(SECOND, a.txn_time, b.txn_time)) <= 300;
```

Q52 [IBM] Write a gaps-and-islands query to find consecutive days of user activity.

```
WITH numbered AS (
  SELECT user_id, activity_date,
    activity_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY activity_date) AS
grp
  FROM (SELECT DISTINCT user_id, activity_date FROM events) t
)
SELECT user_id, MIN(activity_date) streak_start, MAX(activity_date) streak_end,
  COUNT(*) streak_len
FROM numbered GROUP BY user_id, grp ORDER BY user_id, streak_start;
```

The date - ROW_NUMBER trick: for consecutive dates, the difference is constant per group.

Q53 [Amazon] Write a funnel analysis query (impression → click → purchase).

```
SELECT
  COUNT(DISTINCT user_id) impressions,
  COUNT(DISTINCT CASE WHEN action='click' THEN user_id END) clicks,
  COUNT(DISTINCT CASE WHEN action='purchase' THEN user_id END) purchases
FROM events
WHERE event_date = '2024-01-15';
```

Q54 [Databricks] What is a slowly changing dimension Type 4? When is it preferred over Type 2?

SCD Type 4: current attributes in the main dim table; full history in a separate history table. Keeps the main dim small and fast for current-state joins. History table used only when temporal analysis needed.

Prefer over Type 2 when: dim table is queried 95% of the time for current state; history is rarely needed; Type 2 makes the dim table too large for join performance.

Q55 [S&P Global] Write SQL to unpivot columns into rows (wide to long format).

```
-- Unpivot quarterly sales into rows (Snowflake/SQL Server UNPIVOT syntax)
SELECT year, quarter, revenue FROM quarterly_sales
UNPIVOT (revenue FOR quarter IN (Q1, Q2, Q3, Q4)) AS unpvt;

-- Universal: UNION ALL approach
SELECT year, 'Q1' quarter, Q1 revenue FROM quarterly_sales
```

```
UNION ALL SELECT year, 'Q2', Q2 FROM quarterly_sales
UNION ALL SELECT year, 'Q3', Q3 FROM quarterly_sales
UNION ALL SELECT year, 'Q4', Q4 FROM quarterly_sales;
```

Q56 [Meta] Write a query to calculate the 95th percentile response time per API endpoint.

```
SELECT endpoint,
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY response_ms) AS p95_ms
FROM api_logs
WHERE log_date = CURRENT_DATE
GROUP BY endpoint
ORDER BY p95_ms DESC;
```

Q57 [Netflix] How would you calculate DAU, WAU, MAU and the DAU/MAU ratio?

```
SELECT
  COUNT(DISTINCT CASE WHEN event_date = CURRENT_DATE THEN user_id END) DAU,
  COUNT(DISTINCT CASE WHEN event_date >= CURRENT_DATE-6 THEN user_id END) WAU,
  COUNT(DISTINCT CASE WHEN event_date >= CURRENT_DATE-29 THEN user_id END) MAU,
  ROUND(1.0 * COUNT(DISTINCT CASE WHEN event_date = CURRENT_DATE THEN user_id END)
       / NULLIF(COUNT(DISTINCT CASE WHEN event_date>=CURRENT_DATE-29 THEN user_id
END),0),4) dau_mau
FROM user_events;
```

Q58 [Teradata] Write a Teradata-specific query using QUALIFY instead of a subquery for window filtering.

QUALIFY is Teradata/Snowflake extension that filters on window function results without a subquery.

```
-- Standard SQL (subquery required)
SELECT * FROM (SELECT *, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) rk FROM
emp) t WHERE rk=1;

-- Teradata QUALIFY (cleaner)
SELECT * FROM emp
QUALIFY RANK() OVER (PARTITION BY dept ORDER BY salary DESC) = 1;
```

Q59 [Airbnb] Write SQL to compute week-over-week booking growth rate using a self-join.

```
SELECT a.week, a.bookings, b.bookings prev_bookings,
  ROUND(100.0*(a.bookings-b.bookings)/NULLIF(b.bookings,0),2) wow_pct
FROM weekly_bookings a
LEFT JOIN weekly_bookings b ON a.week = b.week + INTERVAL '7 days';
```

2.1 Snowflake / Cloud DW Specific

Q60 [Snowflake] What is a micro-partition in Snowflake? How does it enable pruning?

Snowflake automatically divides tables into micro-partitions (~50-500MB compressed, 16MB uncompressed). Each stores min/max metadata per column.

Pruning: when a query filters col = X, Snowflake reads only micro-partitions whose min/max range overlaps X. No manual partitioning needed.

Q61 [Snowflake] What is clustering in Snowflake and when should you add a cluster key?

Clustering sorts micro-partitions by a key column, improving pruning for range scans on that column.

Add cluster key when: table > 1TB, a specific column is almost always in the WHERE clause, and natural data arrival order does not match query patterns.

```
ALTER TABLE events CLUSTER BY (event_date);
```

Q62 [Snowflake] Explain the difference between COPY INTO and INSERT for loading data.

COPY INTO: bulk load from staged files (S3, Azure Blob, GCS). Highly parallelized, compressed. No transaction overhead. The right way to load data at scale.

INSERT: row-by-row or small batch via SQL. For small volumes or derived data (INSERT INTO ... SELECT).

```
COPY INTO orders FROM @my_s3_stage/orders/2024-01-15/ FILE_FORMAT=(TYPE=PARQUET);
```

Q63 [Snowflake] What is a materialized view in Snowflake? Tradeoffs vs regular view?

Materialized view: precomputed query result stored physically. Auto-refreshed by Snowflake when base data changes.

+ Faster for repeated expensive aggregations.

- Extra storage cost, refresh lag, limited SQL features (no JOINS in some versions).

Use for: expensive aggregations queried frequently, near-real-time dashboards where a few-minute lag is OK.

CHAPTER 3 — PYSPARK & SPARK INTERNALS (Questions 91–150)

3.1 Spark Architecture

Q64 [Databricks] Explain the Spark execution model: Driver, Executor, Stage, Task, Shuffle.

- Driver: JVM process running SparkSession. Builds DAG, schedules tasks, collects results.
- Executor: JVM on worker nodes. Runs tasks, caches data.
- Job: each action triggers one job.
- Stage: set of tasks with no shuffle between them. Wide transformations (groupBy, join) create stage boundaries.
- Task: work on one partition. N partitions = N tasks per stage.
- Shuffle: network exchange of data between stages. Most expensive operation in Spark.

Q65 [Databricks] What is lazy evaluation and why does it matter?

Transformations (filter, select, groupBy, join) are lazy — they just build a logical plan. Nothing runs until an action (count, write, collect, show) is called.

Why it matters: Spark optimizes the full DAG before running. Catalyst optimizer can push down filters, reorder joins, eliminate columns. You get optimization "for free."

Q66 [Netflix] Why is collect() dangerous? What should you use instead?

collect() pulls ALL data from all executors to the Driver. For large DataFrames, this causes Driver OOM.

```
# DANGEROUS on 10GB DataFrame:
rows = df.collect()  # OOM crash

# Safe alternatives:
df.show(20)          # show 20 rows in console
df.take(100)         # take 100 rows to driver (small list)
df.limit(1000).toPandas()  # bounded conversion
df.write.parquet("s3://...")  # write to storage
```

Q67 [IBM] What is the difference between repartition() and coalesce()?

```
# repartition(n): full shuffle. Can increase OR decrease. Even distribution. Expensive.
df.repartition(400)                # evenly spread to 400 partitions
df.repartition(200, "customer_id") # partition by key (good before join on same key)

# coalesce(n): no shuffle. Can ONLY decrease. Merges existing partitions.
df.coalesce(10)  # merge down — use before write to reduce output files
```

Rule: coalesce to reduce, repartition to increase or rebalance.

Q68 [Amazon] How do you tune the number of Spark partitions?

Target: 128MB–200MB per partition. Too small = too many tasks, high overhead. Too large = OOM, can't parallelize.

```
# Check partition sizes
df.rdd.getNumPartitions()  # current partition count

# Rule of thumb: 2-4x CPU cores available to the job
spark.conf.set("spark.sql.shuffle.partitions", "400")  # default is 200

# With AQE (Spark 3+): automatically coalesces small shuffle partitions
spark.conf.set("spark.sql.adaptive.enabled","true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled","true")
```

Q69 [Databricks] What is Adaptive Query Execution (AQE) in Spark 3? What problems does it solve?

AQE re-optimizes the query plan at runtime using actual statistics collected during execution.

1. Coalescing small shuffle partitions (avoids millions of tiny tasks).
2. Converting sort-merge joins to broadcast joins when runtime size < threshold.
3. Skew join optimization: detects and splits skewed partitions automatically.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Q70 [IBM] Explain data spill in Spark. When does it happen and how do you fix it?

Spill occurs when a task's in-memory data exceeds available executor memory — Spark writes excess to disk. Orders of magnitude slower than memory operations.

Causes: too few partitions for the data volume, insufficient executor memory, data skew.

```
# Fix 1: increase partitions
spark.conf.set("spark.sql.shuffle.partitions", "800")

# Fix 2: increase executor memory
--executor-memory 16g --executor-cores 4

# Fix 3: salt skewed keys (see skew section)
```

Q71 [Netflix] What is caching in Spark? When should you cache vs not cache?

```
df.cache()          # cache in memory (MEMORY_AND_DISK default)
df.persist(StorageLevel.MEMORY_ONLY)  # or choose storage level
df.unpersist()      # release cache explicitly
```

Cache WHEN: DataFrame is reused multiple times in the same job (e.g., train/test split, iterative ML).

Do NOT cache: DataFrames used only once (wastes memory), very large DFs that won't fit in memory (causes spill), streaming DataFrames.

Q72 [Databricks] What are Spark storage levels? When would you use MEMORY_AND_DISK_SER?

- MEMORY_ONLY: fastest but data lost if not enough memory.
- MEMORY_AND_DISK: spills to disk if not enough memory. Safe default.
- MEMORY_ONLY_SER: serialized (smaller footprint), slightly slower deserialization.
- MEMORY_AND_DISK_SER: serialized + disk spill. Use for large DFs on memory-constrained clusters.
- DISK_ONLY: only disk, no memory. Rare — just write to storage instead.

Q73 [Amazon] Explain Tungsten execution engine and Catalyst optimizer.

Catalyst: logical query optimizer. Converts SQL/DataFrame to a logical plan, applies rule-based and cost-based optimizations (predicate pushdown, column pruning, join reordering), then generates a physical plan.

Tungsten: execution engine focused on CPU efficiency. Uses code generation (Whole-Stage CodeGen), off-heap memory management, and cache-aware algorithms to reduce JVM overhead.

3.2 PySpark Joins

Q74 [Netflix] Explain broadcast join vs sort-merge join vs shuffle-hash join. How do you force each?

- Broadcast join: small table sent to all executors. No shuffle. Fastest. Auto-triggered when table < autoBroadcastJoinThreshold (10MB).
- Sort-merge join: both sides shuffled on key then sorted and merged. Default for large-large joins. Robust.
- Shuffle-hash join: both sides shuffled; hash table built from smaller side. Faster than sort-merge if one side fits in executor memory.

```
from pyspark.sql.functions import broadcast
df_large.join(broadcast(df_small), "key")   # force broadcast
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50*1024*1024)  # 50MB
```

Q75 [Databricks] What is bucketing in Spark? How does it eliminate shuffle in joins?

Bucketing pre-shuffles and saves data into a fixed number of files (buckets) by a key. When two bucketed tables with the same bucket key and count are joined, Spark skips the shuffle entirely.

```
df.write.bucketBy(64,"user_id").sortBy("user_id").saveAsTable("events_bucketed")
df2.write.bucketBy(64,"user_id").sortBy("user_id").saveAsTable("users_bucketed")
# Now join skips shuffle:
spark.table("events_bucketed").join(spark.table("users_bucketed"),"user_id")
```

Bucketing is permanent (table metadata). Effective for repeated joins on same key.

Q76 [IBM] How do you handle data skew in a Spark join? Explain salting.

Skew: one key value (e.g., customer_id = "MEGA_CORP") has millions of rows — one task handles them all while others finish quickly.

```
from pyspark.sql import functions as F
SALT = 10
# Salt the large table
df_large = df_large.withColumn("salt", (F.rand()*SALT).cast("int")) \
    .withColumn("key_s", F.concat("key", F.lit("_"), "salt"))
# Explode the small table to match all salts
df_small = df_small.withColumn("salt", F.explode(F.array([F.lit(i) for i in
range(SALT)]))) \
    .withColumn("key_s", F.concat("key", F.lit("_"), "salt"))
result = df_large.join(df_small, "key_s").drop("salt","key_s")
```

Q77 [Amazon] What is a semi-join and anti-join in PySpark?

```
# Semi-join: rows from left that have at least one match on right (no right columns
returned)
df_left.join(df_right, "key", "left_semi")

# Anti-join: rows from left with NO match on right
df_left.join(df_right, "key", "left_anti")
```

Useful for filtering DataFrames based on existence in another set.

3.3 PySpark Transformations & Code

Q78 [Netflix] Write PySpark to sessionize a clickstream (session gap = 30 min).

```
from pyspark.sql import functions as F, Window
w = Window.partitionBy("user_id").orderBy("ts")
df = df.withColumn("prev_ts", F.lag("ts").over(w)) \
       .withColumn("gap_min",
          (F.col("ts").cast("long")-F.col("prev_ts").cast("long"))/60) \
       .withColumn("new_sess", (F.col("gap_min")>30)|F.col("prev_ts").isNull()) \
       .withColumn("session_id", F.sum(F.col("new_sess").cast("int")).over(w))
```

Q79 [Databricks] How do you handle schema evolution with mergeSchema in PySpark?

```
# mergeSchema: auto-add new columns from new files
df = spark.read.option("mergeSchema","true").parquet("s3://bucket/events/")

# Delta Lake: schema evolution on write
df.write.format("delta").option("mergeSchema","true").mode("append").save("...")

# enforceSchema: fail if schema doesn't match (strict mode)
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled","false")
```

Q80 [IBM] What are PySpark UDFs? Why should you avoid them and what are the alternatives?

UDFs serialize data from JVM to Python, apply function, serialize back. This serialization overhead is ~10x slower than native Spark functions.

UDFs are also opaque to Catalyst — no predicate pushdown, no code generation.

```
# Avoid:
@F.udf(returnType=StringType())
def clean(x): return x.strip().lower()

# Prefer native functions:
df.withColumn("clean_col", F.trim(F.lower(F.col("col"))))

# When UDF is unavoidable, use Pandas UDF (vectorized, 10x faster):
@F.pandas_udf(returnType=DoubleType())
def score(series: pd.Series) -> pd.Series:
    return series.apply(my_model.predict)
```

Q81 [Amazon] Write PySpark to compute the median of a column (no approx).

```
from pyspark.sql.functions import percentile_approx, expr
```

```
# Exact median (expensive — full sort):
df.selectExpr("percentile(amount, 0.5) AS median_amount")

# Approximate (faster, configurable accuracy):
df.select(F.percentile_approx("amount", 0.5, 100000).alias("approx_median"))
```

Q82 [Netflix] How do you read only specific partitions of a parquet dataset in PySpark?

```
# Partition filter pushed down to file system scan (reads only matching dirs)
df = spark.read.parquet("s3://bucket/events/") \
    .filter(F.col("event_date") >= "2024-01-01")  # partition column: prunes dirs

# Or use path directly
df = spark.read.parquet("s3://bucket/events/event_date=2024-01-15/")

# Nested: year/month/day partitioning
df = spark.read.parquet("s3://bucket/events/year=2024/month=01/")
```

Q83 [IBM] Explain foreachBatch in Structured Streaming. When is it used?

foreachBatch: in each micro-batch, Spark calls your function with the batch DataFrame and batch ID. You can then apply arbitrary batch operations (MERGE, deduplication, multi-sink writes) that aren't native streaming operations.

```
def write_batch(df, epoch_id):
    df.dropDuplicates(["event_id"]) \
      .write.format("delta").mode("append").save("s3://delta/events")

stream.writeStream.foreachBatch(write_batch) \
    .option("checkpointLocation","s3://ckpt/events") \
    .start()
```

Q84 [Databricks] What is Delta Live Tables (DLT)? How does it differ from plain Spark + Delta?

DLT is a declarative pipeline framework in Databricks. You define tables with @dlt.table / LIVE TABLE SQL; DLT handles orchestration, retries, data quality checks, and lineage automatically.

- Auto-manages dependencies between tables in a pipeline.
- Built-in expectations (data quality assertions).
- Incremental updates handled automatically.

Plain Spark + Delta: full control, more code, manage orchestration yourself with Airflow/Databricks Workflows.

Q85 [Amazon] How do you write unit tests for PySpark transformations?

```
import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder.master("local[2]").appName("test").getOrCreate()

def test_dedup(spark):
    data = [(1,"a"),(1,"a"),(2,"b")]
    df = spark.createDataFrame(data, ["id","val"])
    result = df.dropDuplicates(["id"])
    assert result.count() == 2
    assert sorted(result.toPandas()["id"].tolist()) == [1,2]
```

Use local[2] SparkSession in tests — no cluster needed. Keep transformations as pure functions for testability.

Q86 [Netflix] What is Z-ordering in Delta Lake? How does it improve query performance?

Z-ordering multi-dimensionally clusters data within Parquet files using a Z-order (Morton) curve. Columns with similar values are co-located, improving micro-partition pruning on multiple columns simultaneously.

```
OPTIMIZE delta.`s3://bucket/delta/events`
ZORDER BY (user_id, event_date);
```

Queries filtering on user_id AND event_date read far fewer files. Works at file level within partitions.

Q87 [Databricks] Explain the difference between append, overwrite, and overwritePartitions write modes.

```
df.write.mode("append")     # adds data; never deletes existing files
```

```
df.write.mode("overwrite")  # deletes entire table/path then writes fresh (DANGEROUS on
partitioned tables)

# Safe partial overwrite: only touch affected partitions
spark.conf.set("spark.sql.sources.partitionOverwriteMode","dynamic")
df.write.mode("overwrite").partitionBy("event_date").save("...")
```
Dynamic overwrite: only partitions present in df are overwritten; others untouched.

CHAPTER 4 — PIPELINES, AIRFLOW & KAFKA (Questions 151–210)

4.1 Pipeline Design

Q88 [Netflix] What is the difference between batch, micro-batch, and streaming pipelines?

• Batch: process data collected over a period (hourly/daily). High latency (minutes to hours). Simple. Tools: Spark, Hive, dbt.

• Micro-batch: small batches every few seconds to minutes. Spark Structured Streaming default. Good balance of latency vs complexity.

• Streaming: event-by-event, sub-second latency. True stream processing. Tools: Flink, Kafka Streams. Most complex.

Q89 [IBM] What is idempotency in pipelines? How do you implement it?

Idempotency: running the same pipeline multiple times produces identical results. Critical for safe retries.

```
# Idempotent write: overwrite partition
spark.conf.set("spark.sql.sources.partitionOverwriteMode","dynamic")
df.write.mode("overwrite").partitionBy("event_date").save("s3://...")

# Idempotent MERGE (upsert)
delta_table.merge(source_df,"target.id = source.id") \
    .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
```

Avoid APPEND mode in rerunnable pipelines — it causes duplicates on retry.

Q90 [Amazon] Explain exactly-once, at-least-once, and at-most-once delivery semantics.

• At-most-once: fire and forget. May lose messages. No retry. Use for non-critical metrics.

• At-least-once: retry on failure. May duplicate. Requires idempotent consumers. Default in most systems.

• Exactly-once: hardest guarantee. Requires idempotent producer + transactional consumer coordination (Kafka transactions). High overhead.

In practice: design for at-least-once delivery with idempotent writes. This achieves the same outcome at lower cost.

Q91 [Databricks] What is the Lambda Architecture? What is Kappa Architecture? Why is Kappa preferred today?

Lambda: batch layer (accurate, slow) + speed layer (fast, approximate). Two codebases. Complex to maintain.

Kappa: single streaming path. Replay Kafka log for historical reprocessing. One codebase.

Kappa preferred because: streaming engines (Flink, Spark Streaming) now match batch accuracy; maintaining two systems doubles engineering burden; reprocessing via log replay is clean.

Q92 [Netflix] What is backpressure in streaming pipelines? How does Spark Structured Streaming handle it?

Backpressure: consumer is slower than producer, causing buffer overflow or lag buildup.

Spark Structured Streaming handles it with maxOffsetsPerTrigger and maxFilesPerTrigger — limits how much data each micro-batch processes, preventing OOM and lag cascades.

```
stream = spark.readStream.format("kafka")
    .option("maxOffsetsPerTrigger", 100000)
    .load()
```

Q93 [S&P Global] How do you handle late-arriving data in streaming pipelines?

• Watermarking: set maximum expected lateness. Events beyond watermark are dropped.

• Event time windowing: use event_time column, not processing time, for windows.

```
stream.withWatermark("event_time","2 hours")  # tolerate 2hr late arrival
    .groupBy(F.window("event_time","1 hour"),"user_id")
    .agg(F.count("*"))
```

Alternatively: reprocess with batch job nightly to correct with late-arriving records.

Q94 [IBM] What is Change Data Capture (CDC)? Compare log-based vs timestamp-based CDC.

CDC captures row-level changes (INSERT/UPDATE/DELETE) from a source database and streams them downstream.

• Log-based (Debezium, GoldenGate): reads DB transaction logs. Zero query impact on source. Captures DELETEs. Best choice.

• Timestamp-based: query WHERE updated_at > last_run. Simple, but misses DELETEs; clock drift risk; misses rows that were updated multiple times between runs.

• Trigger-based: DB triggers on change. Adds write overhead. Captures all changes.

Q95 [Netflix] How do you design a pipeline that is both fault-tolerant and resumable?

1. Checkpoint state: Spark Structured Streaming writes checkpoint to durable storage (S3). On restart, resumes from last committed offset.

2. Idempotent writes: MERGE / partition overwrite prevents duplicate records on reprocessing.

3. Dead letter queue: unprocessable records go to a separate topic/path for manual review.

4. Retry with backoff: transient failures (network, throttle) retried with exponential backoff.

5. Alerting on lag: Kafka consumer lag alert triggers on-call before lag grows uncontrollably.

Q96 [Amazon] What is the small files problem in data lakes? How do you solve it?

Small files: millions of tiny Parquet files (KB-sized) cause: massive S3 LIST API calls, high task overhead, slow query planning.

Root causes: streaming micro-batches write many small files; over-partitioning.

```
# Fix 1: coalesce before write
df.coalesce(10).write.parquet("s3://...")

# Fix 2: Delta OPTIMIZE (compaction)
spark.sql("OPTIMIZE delta.`s3://bucket/events/`")

# Fix 3: increase micro-batch trigger interval
stream.trigger(processingTime="5 minutes")
```

4.2 Airflow

Q97 [General] Explain all key Airflow concepts: DAG, Operator, Task, XCom, Sensor, TriggerRule.

• DAG: Directed Acyclic Graph. Defines tasks and their dependencies. Scheduled by a cron expression.

• Operator: defines WHAT a task does. PythonOperator, BashOperator, SparkSubmitOperator, EmrCreateJobFlowOperator...

• Task: running instance of Operator in a specific DAG run.

• XCom: key-value store for inter-task communication. Use for small values (<48KB). Not for large data.

• Sensor: polls external condition (file exists, Kafka partition ready). Blocks until True.

• TriggerRule: when to trigger. Defaults: all_success. Others: all_done, one_success, none_failed, all_failed.

Q98 [IBM] What is the difference between schedule_interval and start_date? What does catchup do?

start_date: earliest date the DAG can run for. schedule_interval: how often (cron or timedelta).

catchup=True: if start_date is in the past, Airflow creates backfill runs for every missed interval. catchup=False: only runs from "now" forward.

```
with DAG("my_dag", start_date=datetime(2024,1,1),
         schedule_interval="0 6 * * *", catchup=False) as dag:
```

Q99 [Netflix] How do you implement dynamic DAG generation in Airflow?

Generate DAGs programmatically in a Python file using a loop. Airflow parses the file and creates one DAG object per iteration.

```
for env in ["dev","staging","prod"]:
    dag_id = f"pipeline_{env}"
    with DAG(dag_id, schedule_interval="@daily",
             start_date=datetime(2024,1,1), catchup=False) as dag:
        t = PythonOperator(task_id="run",
            python_callable=run_pipeline, op_kwargs={"env":env})
```

```
    globals()[dag_id] = dag  # register in Airflow
```

Q100 [Amazon] What is a Sensor in Airflow? What is the difference between poke and reschedule mode?

poke mode: sensor holds a worker slot and checks condition every poke_interval seconds. Wastes a slot while waiting.

reschedule mode: sensor releases the worker slot between checks. Sleeps and reschedules itself. Much more resource-efficient for long waits.

```
from airflow.sensors.s3_key_sensor import S3KeySensor
S3KeySensor(task_id="wait", bucket_name="raw", bucket_key="data/{{ ds }}/_SUCCESS",
    mode="reschedule", poke_interval=300, timeout=3600)
```

Q101 [IBM] How do you pass data between tasks in Airflow? What are XCom limitations?

```
# Push to XCom
def my_task(**context):
    context["ti"].xcom_push(key="row_count", value=42000)

# Pull in downstream task
def downstream(**context):
    count = context["ti"].xcom_pull(task_ids="my_task", key="row_count")
```

Limitations: XCom is stored in Airflow metadata DB. Not for large data (> a few KB). For large data, write to S3/GCS and pass the path as XCom.

Q102 [Databricks] What strategies do you use to make Airflow DAGs production-grade?

- Retries + retry_delay: handle transient failures.
- SLA alerts: miss_sla triggers email/Slack.
- Idempotent tasks: safe to re-run without side effects.
- Parameterization via Variables / Connections: no hardcoded credentials or paths.
- Dedicated pools: limit concurrency for resource-constrained tasks.
- Unit tests: test task logic independently with PythonOperator.
- DAG-level timeouts: dagrun_timeout prevents zombie runs.

Q103 [Netflix] How do you backfill 2 years of data in Airflow without killing production?

```
# CLI backfill with concurrency limit
airflow dags backfill my_dag -s 2022-01-01 -e 2024-01-01 --max-jobs 3
```

Additional controls: separate resource pool for backfill tasks; run during off-peak hours; throttle with max_active_runs=2 on the DAG; monitor source system load.

4.3 Kafka

Q104 [General] Explain Kafka architecture: Topic, Partition, Broker, Consumer Group, Offset, Replica.

- Topic: named feed. Messages are published to topics.
- Partition: ordered immutable log within a topic. Unit of parallelism and ordering guarantee (ordered within partition, not across).
- Broker: Kafka server. Each partition has a leader broker and N-1 replica brokers.
- Consumer Group: group that collectively consumes a topic. Each partition consumed by exactly one consumer per group. Multiple groups = independent consumption of same data.
- Offset: integer position of a message in a partition. Consumers commit offsets to track progress.
- Replication Factor: copies of each partition. RF=3 means survive 2 broker failures.

Q105 [IBM] What is a Kafka consumer group? How does partition assignment work?

Each partition is assigned to exactly one consumer in the group. More consumers than partitions = idle consumers. Optimal: consumers = partitions.

Rebalance: triggered when consumer joins/leaves. During rebalance, consumption pauses briefly.

Use sticky assignor (partition.assignment.strategy=CooperativeStickyAssignor) for incremental rebalance to minimize disruption.

Q106 [Netflix] What is Kafka retention? How do topics handle log compaction?

Retention: messages kept for a time (default 7 days) or size limit. Consumers can replay from any offset within retention window.

Log compaction: Kafka keeps the LATEST value per key indefinitely. Older records with same key deleted. Useful for a changelog or state store topic where only latest value per key matters (e.g., user profile updates).

```
# Enable log compaction on a topic
kafka-topics.sh --create --topic user-profiles --config cleanup.policy=compact
```

Q107 [Amazon] How do you guarantee message ordering in Kafka?

Kafka guarantees ordering WITHIN a partition, not across partitions.

For strict ordering on an entity (e.g., user_id): always route messages for the same user_id to the same partition using key-based partitioning.

```
# Java producer: set key = user_id
producer.send(new ProducerRecord<>("events", userId, eventJson));
```

To guarantee global ordering: use a single partition (sacrifices parallelism — only for very low volume).

Q108 [IBM] What is idempotent producer in Kafka? How does exactly-once work?

Idempotent producer: Kafka assigns producer ID + sequence number per message. Broker deduplicates retries with same sequence. Prevents duplicate messages on producer retry.

Exactly-once: idempotent producer + Kafka transactions. Producer wraps multiple topic writes in an atomic transaction. Consumer uses isolation.level=read_committed to see only committed data.

```
enable.idempotence=true
transactional.id=my-producer-1
```

Q109 [Netflix] How do you monitor Kafka health? What metrics matter?

- Consumer lag: messages in partition - consumer committed offset. Alert if lag > threshold.
- Under-replicated partitions: partitions not fully replicated. Alert on any URP > 0.
- Broker disk usage: topics filling disk. Alert at 75%.
- Producer error rate: failed sends per second.
- Throughput: bytes in/out per sec, messages per sec.

Tools: Kafka JMX metrics → Prometheus + Grafana, or Confluent Control Center, or Datadog Kafka integration.

CHAPTER 5 — SYSTEM DESIGN (Questions 211–250)

Q110 [Netflix] Design a data platform to process 1 billion events per day. Walk through all layers.

Ingest: Kafka (1000 partitions, RF=3, 7-day retention). Clients → API gateway → Kafka producers.

Schema: Confluent Schema Registry (Avro). Enforce schema at ingest.

Stream: Flink for real-time aggregations (concurrent viewers, fraud signals). Spark Streaming for heavier ETL.

Storage: S3 + Delta Lake. Bronze (raw), Silver (clean), Gold (aggregated).

Catalog: AWS Glue / Hive Metastore for table discovery.

Serve: Snowflake/Redshift for SQL analytics. Druid for low-latency OLAP. Redis for real-time counters.

Orchestrate: Airflow for batch DAGs. Databricks Workflows for Spark jobs.

Monitor: Prometheus + Grafana. PagerDuty for SLA alerts. Great Expectations for DQ.

Q111 [S&P Global] Design a real-time financial market data pipeline with <1 second end-to-end latency.

Feed: FIX/WebSocket market data → custom adapter → Kafka (separate topic per asset class).

Process: Flink (stateful streaming): tick-by-tick OHLCV bars, VWAP, moving averages. RocksDB state.

Checkpoint: Flink checkpoints to S3 every 30 seconds for fault tolerance.

Sink: Cassandra (wide-row per instrument+date for tick history). Redis for latest price (pub/sub). S3+Delta for analytics.

DQ: schema validation at ingest. Price spike detector (alert on >5σ move). Sequence gap detection.

Compliance: immutable raw data store (WORM S3 bucket) for SEC 17a-4 requirements.

Q112 [Amazon] Design an ETL pipeline for an e-commerce company: orders, products, customers → analytics DW.

Sources: PostgreSQL (orders, customers), MongoDB (products), Stripe (payments).

Ingest: Fivetran/Airbyte for managed connectors → S3 staging. Debezium for CDC on PostgreSQL.

Transform: dbt on Snowflake. Silver models: dim_customer, dim_product, fact_orders.

Schedule: Airflow DAG. Source sensors detect when new data arrives → trigger dbt run.

DQ: dbt tests (not_null, unique, referential integrity). Row count alerts. Freshness checks.

Serve: Snowflake for BI (Tableau, Looker). dbt docs for data catalog.

Q113 [IBM] Design a data pipeline for GDPR compliance: right to deletion, right to access.

Right to Deletion (Right to Be Forgotten):

• Pseudonymize PII at ingest (SHA-256 hash + salt). Store mapping in a secure vault.

• On deletion request: delete vault entry. Hash becomes meaningless — all records pseudonymized with that hash are effectively anonymized.

• Delta Lake DELETE or time travel rewind to before the record existed.

Right to Access:

• Maintain a data inventory: which tables/columns contain PII (column tagging in Glue/Purview).

• Query all tables tagged with that user's ID and export within 30 days.

Also: encrypt at rest (KMS), column-level masking for analysts, audit logs of all PII access.

Q114 [Databricks] Design a feature store for ML. What are the challenges?

Offline store: Delta Lake / Parquet on S3. Stores historical features for training. Batch computed.

Online store: Redis / DynamoDB. Serves features at inference time with <10ms latency.

Consistency: offline and online must use same feature computation logic (avoid training-serving skew).

Challenges:

• Point-in-time correctness: training must use features as they existed at prediction time (no future leakage).

• Backfilling: computing historical features efficiently at scale.

• Feature versioning: models depend on specific feature versions.

Tools: Feast, Tecton, Databricks Feature Store.

Q115 [Netflix] Design a content recommendation pipeline: from user events to recommendations.

Capture: Kafka topic "user-watch-events" (user_id, content_id, watch_pct, ts).

Batch features: daily Spark job → compute per-user watch history, genre affinity, completion rates → write to feature store.

Train: weekly ALS/deep learning model on user-content interaction matrix.

Serve: model artifacts deployed to serving layer. Feature store online lookup at request time.

Near-real-time: stream recent watch events → update user feature vector in Redis (recency signals).

A/B test: route % of users to new model. Track CTR and watch time as guardrail metrics.

Q116 [IBM] When would you choose a relational DB vs NoSQL vs a data warehouse vs a data lake?

• Relational DB (PostgreSQL, MySQL): OLTP, complex relations, ACID, <10TB. Normalized.

• NoSQL (Cassandra, DynamoDB): massive scale, simple access patterns, write-heavy, flexible schema, no JOINs.

• Data Warehouse (Snowflake, Redshift, BigQuery): OLAP, structured analytics, SQL-first, pre-modeled.

• Data Lake (S3+Delta, ADLS+Delta): raw/semi-structured, schema-on-read, cheap storage, all formats. Source of truth for all data.

Modern answer: Lakehouse = Data Lake + DW capabilities (Delta Lake, Iceberg, Hudi).

Q117 [Amazon] Explain the CAP theorem in the context of distributed data systems.

CAP: a distributed system can guarantee at most 2 of 3 properties:

• Consistency (C): every read sees the latest write.

• Availability (A): every request gets a response (not necessarily latest).

• Partition Tolerance (P): system keeps working when network partitions occur.

Network partitions are unavoidable in distributed systems. So you choose CP or AP:

• CP (consistent): Zookeeper, HBase. May refuse requests during partition.

• AP (available): Cassandra, DynamoDB. May return stale data during partition.

Choose AP for user-facing systems where latency matters. CP for financial systems where correctness matters.

Q118 [Databricks] What is the Medallion Architecture? Detail Bronze, Silver, and Gold layers.

• Bronze (Raw): data as-is from source. Append-only. Schema-on-read. No transformations. Keep forever. The audit/replay layer.

• Silver (Conformed): deduplicated, schema enforced, NULLs handled, joins applied. Cleaned, validated data. Conformed types.

• Gold (Business): aggregated, business logic applied, star schema / metric tables. Directly consumed by BI tools and ML.

Benefits: incremental refinement, full history in bronze, clean queries in gold, easy debugging (trace gold issue back to bronze).

Q119 [S&P Global] How would you migrate a 50TB on-premise Teradata DW to Snowflake?

1. Assessment: catalog all objects (tables, views, macros, stored procs). Identify Teradata-specific syntax (BTEQ, TD-specific functions).

2. Schema migration: auto-convert DDL (Snowflake migration tools). Adjust Teradata types (BYTEINT→TINYINT, etc.).

3. SQL translation: convert Teradata SQL → Snowflake SQL. Key differences: QUALIFY, DATE arithmetic, SAMPLE, RANK/TITLE.

4. Data migration: Teradata TPT export → S3 → Snowflake COPY INTO. Run in batches by table/partition.

5. Validation: row count + aggregate checksums per table.

6. Cutover: dual-write period, validate reports, cutover BI tools, decommission Teradata.

CHAPTER 6 — TOOLS: dbt, DELTA, TERADATA, CLOUD (Questions 251–280)

6.1 dbt

Q120 [General] What is dbt and what does it replace in the traditional ETL stack?

dbt handles the T (Transform) in ELT. You write SELECT statements; dbt compiles them to DDL/DML, manages dependencies, runs tests, and generates lineage documentation.

Replaces: hand-written stored procedures, manual table creation scripts, undocumented transformation logic. Does NOT replace ingestion (Fivetran/Airbyte handles E+L).

Q121 [IBM] What are the 4 dbt materialization types and when do you use each?

- view: compiled as SQL view. No data stored. Always fresh. Use for simple/cheap queries.
- table: recreated from scratch each run. Use for complex queries run infrequently.
- incremental: appends or merges only new/changed data. Use for large tables where full refresh is too slow.
- ephemeral: CTE injected into downstream models. No object created in DB. Use for intermediate logic reuse.

Q122 [Netflix] Write a dbt incremental model with a unique_key.

```
{{ config(materialized="incremental", unique_key="order_id",
incremental_strategy="merge") }}

SELECT order_id, customer_id, amount, status, updated_at
FROM {{ source("raw","orders") }}

{% if is_incremental() %}
  WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

unique_key tells dbt to MERGE (upsert) on that key — avoiding duplicates on rerun.

Q123 [Amazon] What are dbt tests? Name all schema test types.

Generic tests defined in schema.yml:

- not_null: column has no NULL values.
- unique: column has no duplicate values.
- accepted_values: column values are within a defined list.
- relationships: referential integrity check (FK exists in parent table).

Custom data tests: SQL files in tests/ folder. Any query that returns rows = test failure.

```
# schema.yml
columns:
  - name: order_id
    tests: [unique, not_null]
  - name: status
    tests: [{accepted_values: {values: [pending,confirmed,cancelled]}}]
```

Q124 [Databricks] How do you handle cross-database references in dbt?

```
-- ref() for models within the project
FROM {{ ref("stg_orders") }}

-- source() for raw/external tables
FROM {{ source("raw_db", "orders") }}

-- Cross-project (dbt Mesh): use cross-project ref
FROM {{ ref("analytics", "dim_customer") }}
```

Q125 [IBM] What are dbt macros? Write a macro to standardize column naming.

```
-- macros/clean_name.sql
{% macro clean_column_name(col) %}
    LOWER(TRIM(REPLACE({{ col }}, " ", "_")))
```

```
{% endmacro %}

-- Usage in model
SELECT {{ clean_column_name("product name") }} AS product_name FROM products
```

Macros are Jinja functions. Use for: reusable SQL logic, environment-specific config, generating repetitive SQL.

6.2 Delta Lake & Iceberg

Q126 [Databricks] What is Delta Lake? Explain the transaction log.

Delta Lake: open storage format on Parquet + a _delta_log/ directory of JSON commit files.

Transaction log: every write (INSERT/UPDATE/DELETE/MERGE/OPTIMIZE) appends a new JSON entry recording: files added/removed, schema, timestamp, operation details.

ACID: atomicity (single log commit), isolation (optimistic concurrency — snapshot isolation), durability (log on durable storage).

Q127 [IBM] Explain Delta Lake time travel. How far back can you go?

```
-- Read version 5
df = spark.read.format("delta").option("versionAsOf",5).load("s3://...")

-- Read as of timestamp
df = spark.read.format("delta").option("timestampAsOf","2024-01-15").load("s3://...")

-- SQL
SELECT * FROM events VERSION AS OF 5;
SELECT * FROM events TIMESTAMP AS OF '2024-01-15 00:00:00';
```

How far back: as long as old Parquet files are not removed by VACUUM. Default VACUUM retention = 7 days.

Q128 [Netflix] What is Apache Iceberg? How does it differ from Delta Lake?

Both are open table formats (OTF) adding ACID + schema evolution + time travel to Parquet on object storage.

• Delta Lake: Databricks-originated. Native Spark integration. Delta Sharing for data exchange.

• Iceberg: Apache project. Engine-agnostic (Spark, Flink, Trino, Hive, Dremio). Better partition evolution. Row-level deletes via equality deletes.

Iceberg is preferred in multi-engine environments. Delta is preferred in Databricks-centric stacks.

Q129 [Amazon] Explain OPTIMIZE and VACUUM in Delta Lake and their risks.

OPTIMIZE: compacts small files into larger files (target ~1GB). Can add Z-ordering. Improves read performance.

VACUUM: removes old Parquet files no longer referenced by the transaction log. Frees storage.

```
delta_table.vacuum(168)  # 168 hours = 7 days minimum retention
```

Risk: VACUUM with retention < 7 days can remove files that concurrent long-running reads still need, causing FileNotFoundException.

Never set VACUUM retention below 7 days in production.

6.3 Teradata-Specific

Q130 [Teradata] What is a Primary Index in Teradata? Difference between UPI and NUPI?

PI determines which AMP stores each row (hash of PI value → AMP). Critical for data distribution.

• UPI (Unique PI): enforces uniqueness. Each PI value → exactly one AMP. Row retrieval in one AMP access.

• NUPI (Non-Unique PI): allows duplicates. May cause skew if a value is extremely common.

Choosing PI: use the most frequently joined column. Bad PI = data skew = one AMP overloaded.

Q131 [Teradata] What is a Secondary Index in Teradata? When do you use it?

SI creates a subtable mapping SI column value → row ID + AMP location. Allows row retrieval by non-PI column without full table scan.

• USI (Unique Secondary Index): O(2 AMP) retrieval for unique column.

• NUSI (Non-Unique): scans subtable rows. May still require multiple AMP accesses.

Use SI when a column is frequently in WHERE clause but is not the PI. Cost: additional storage + write overhead.

Q132 [Teradata] Explain SET vs MULTISET tables and their performance implications.

SET: Teradata default. Enforces row uniqueness by checking for duplicate rows on every insert (AMP-level comparison). Safe but expensive for large bulk loads.

MULTISET: allows duplicate rows. No uniqueness check on insert. Significantly faster loads.

Best practice for DW: use MULTISET + explicit primary/unique indexes for uniqueness constraints. Avoid SET table for large fact tables.

Q133 [Teradata] What are the Teradata loading tools? When do you use each?

• FastLoad: bulk load into EMPTY tables only. Bypasses transient journal. Fastest for initial loads.

• MultiLoad: load into existing tables. Supports INSERT/UPDATE/DELETE/UPSERT. 5 tables max concurrently.

• FastExport: parallel export from Teradata to flat files.

• BTEQ: interactive SQL. Good for scripts, not bulk loads.

• TPT (Teradata Parallel Transporter): unified framework. All operations. Best for modern pipelines.

• Teradata JDBC/ODBC: row-by-row. Only for small volumes or ad-hoc.

Q134 [Teradata] What is COLLECT STATISTICS and why is it critical?

COLLECT STATISTICS gathers column demographics (min, max, cardinality, nulls, histogram) and stores them for the query optimizer.

Without current stats: optimizer guesses row counts → bad execution plans → slow queries.

```
COLLECT STATISTICS COLUMN(customer_id) ON orders;
COLLECT STATISTICS COLUMN(order_date) ON orders;
COLLECT STATISTICS INDEX(order_id) ON orders;
```

Run after: initial load, after large data changes, if query plans suddenly degrade.

Q135 [Teradata] What is the EXPLAIN command in Teradata? What do you look for?

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 123;
```

Look for:

• "We do an all-AMPs retrieve" = full table scan (bad for large tables).

• "We do a one-AMP retrieve" = PI-based single AMP access (optimal).

• "by way of a single-AMP step" = SI being used.

• Row count estimates: if wildly off, COLLECT STATISTICS needed.

• Spool usage: temporary tables in Teradata. Excessive spool = performance issue.

CHAPTER 7 — DATA QUALITY, GOVERNANCE & SECURITY (Questions 281–310)

Q136 [IBM] What are the 6 dimensions of data quality?

1. Completeness: are required fields populated? No unexpected NULLs.
2. Validity: do values conform to defined rules (format, range, type)?
3. Uniqueness: no duplicate records on key columns.
4. Consistency: same data matches across systems (e.g., total in orders = sum of order_lines).
5. Timeliness: data arrives and is processed within SLA.
6. Accuracy: data correctly represents the real-world entity.

Q137 [Netflix] How do you implement automated data quality checks in a Spark pipeline?

```
import great_expectations as ge

df_ge = ge.from_pandas(df.toPandas())
df_ge.expect_column_values_to_not_be_null("order_id")
df_ge.expect_column_values_to_be_unique("order_id")
df_ge.expect_column_values_to_be_between("amount", min_value=0, max_value=1e6)
df_ge.expect_column_values_to_be_in_set("status",["pending","confirmed","cancelled"])

result = df_ge.validate()
if not result["success"]:
    raise ValueError(f"DQ failed: {result}")
```

Alternatively: write custom Spark SQL assertions that raise on failure.

Q138 [Amazon] How do you detect and alert on data drift?

Statistical drift: track mean, stddev, null rate, cardinality per column. Alert on >3σ deviation from 30-day baseline.

Distribution drift: Population Stability Index (PSI). PSI < 0.1 = stable. 0.1-0.2 = moderate change. > 0.2 = significant drift.

Volume drift: daily row count deviating >20% from rolling 7-day average → alert before downstream jobs run.

Tools: Monte Carlo, Bigeye, Anomalo, Great Expectations, custom Spark + Prometheus.

Q139 [S&P Global] How do you handle PII in a data lake? Explain masking, tokenization, and pseudonymization.

• Masking: irreversible obfuscation (john@e.com → j***@e***.com). For analytics teams who don't need real values.

• Tokenization: replace PII with random token stored in a secure vault. Reversible via vault lookup.

• Pseudonymization: consistent hash (SHA-256 + salt). Same PII → same pseudonym. Allows joins without exposing PII.

```
df = df.withColumn("user_id_pseudo",
    F.sha2(F.concat(F.col("email"), F.lit("SECRET_SALT")), 256))
```

Q140 [IBM] What is data lineage and why does it matter?

Data lineage tracks how data flows from source to destination: where it came from, what transformations were applied, who consumed it.

Why it matters: root cause analysis (which upstream source caused a downstream anomaly?), compliance (show regulators data provenance), impact analysis (if I change table X, what breaks?), debugging failed pipelines.

Tools: Apache Atlas, OpenLineage (Marquez), dbt docs (model lineage graph), Databricks Unity Catalog.

Q141 [Netflix] What is a data catalog? How does it differ from a data dictionary?

Data dictionary: static documentation of schema, column definitions, data types. Usually a spreadsheet or wiki.

Data catalog: active, searchable platform that combines metadata, lineage, data profiling, access controls, and search. Auto-updates from sources.

Tools: AWS Glue Data Catalog, Google Data Catalog, Databricks Unity Catalog, Apache Atlas, Alation, Collibra.

Q142 [Amazon] Explain row-level and column-level security in a data warehouse.

Row-level: users see only rows matching their permissions (e.g., EMEA analyst sees only EMEA data).

Column-level: mask or hide specific columns based on role (analysts see masked email, engineers see real email).

```
-- Snowflake: row access policy
CREATE ROW ACCESS POLICY region_policy AS (region STRING) RETURNS BOOLEAN ->
  CURRENT_ROLE() = 'ADMIN' OR region = CURRENT_USER();
ALTER TABLE orders ADD ROW ACCESS POLICY region_policy ON (region);

-- Column masking (see Ch 7 earlier)
```

Q143 [S&P Global] What is a data contract? Why are they important?

A data contract is a formal agreement between data producer and consumer defining: schema, field types, nullable fields, freshness SLA, volume expectations, versioning policy.

Why important: prevents "silent breaking changes" — producer changes a column type and downstream pipelines break. With contracts, changes require version bump and consumer notification.

Tools: dbt contracts (enforced schema), DataHub/Atlan for contract registry, OpenDataMesh.

Q144 [IBM] How do you ensure data pipeline SLA compliance? What happens when SLA is missed?

Define SLA: "Gold layer must be available by 8 AM UTC daily."

Monitor: Airflow SLA callbacks, Datadog/Prometheus on task duration, freshness checks on target tables.

Alert: PagerDuty/Slack alert when SLA at risk (e.g., task running longer than 80% of budget).

Escalation: on-call runbook for common failures (source delay, cluster resource issue, data quality failure).

Post-miss: notify downstream consumers immediately. Estimate ETL. Provide interim workaround if possible.

Q145 [Databricks] What is Unity Catalog in Databricks? What governance capabilities does it provide?

Unity Catalog: centralized governance layer for all data and AI assets in Databricks.

- Three-level namespace: catalog.schema.table
- Fine-grained access control: GRANT/REVOKE on tables, columns, rows
- Data lineage: automated column-level lineage across all Databricks workloads
- Audit logs: who accessed what, when
- Attribute-based access control (ABAC): tag columns as PII, auto-apply masking policies

CHAPTER 8 — BEHAVIORAL & SCENARIO QUESTIONS (Questions 311–330)

Q146 [General] Tell me about a time you fixed a critical data pipeline failure in production.

Use STAR: Situation (what failed, business impact), Task (your role), Action (exact steps: detect→diagnose→fix→communicate→document), Result (SLA restored, prevention added).

Template: "Our daily revenue reconciliation pipeline failed silently for 2 days. I detected it via a data freshness alert, traced root cause to a schema change in an upstream Postgres table (new NOT NULL column without migration), patched the Spark job to handle the new schema with a COALESCE default, added schema validation as a DAG gate, and wrote a runbook. Finance was unblocked within 3 hours. We added a dbt test on source schema change detection."

Q147 [General] How do you prioritize when multiple pipelines fail simultaneously?

1. Triage by business impact: revenue reporting > finance SLA > marketing > internal analytics.
2. Communicate immediately to stakeholders — set ETAs early.
3. Quick wins first: a 5-min config fix that unblocks a critical pipeline beats a 2-hour deep dive on a low-priority one.
4. Parallel mitigation: can I give stakeholders a manual workaround while fixing root cause?
5. Blameless post-mortem after: timeline, root cause, action items.

Q148 [Netflix] You discover a metric has been wrong for 6 months. What do you do?

1. Do NOT fix silently. Communicate to stakeholders immediately with impact scope.
2. Quantify: by how much, for which dates, which reports/decisions were affected.
3. Fix forward: correct the calculation logic.
4. Backfill historical data with corrected values.
5. Validate: reconcile corrected numbers against a known-good source.
6. Root cause: why did automated tests not catch this? Add the missing test.
7. Blameless post-mortem: process improvement focus.

Q149 [IBM] How do you work with data scientists who need data you haven't built yet?

- Clarify requirements: what exactly is needed? What schema? What grain? What freshness?
- Offer a provisional data mart with documented caveats while building the production version.
- Agree on a data contract upfront: schema, SLA, quality expectations.
- Involve them in review of the model/transformation before build — prevents rework.
- Timebox: prioritize if their model training deadline is a hard business constraint.

Q150 [Amazon] How do you approach technical debt in data pipelines?

- Log and track debt in your team's backlog (not just mentally). Make it visible.
- Classify: critical (reliability risk) vs important (maintainability) vs nice-to-have.
- Advocate for 20% sprint capacity dedicated to tech debt (like Spotify/Netflix engineering norms).
- Prioritize debt that creates on-call burden — unreliable pipelines that page people at 3 AM are business problems, not just tech debt.
- Refactor incrementally with tests to prove behavior is unchanged.

Q151 [Databricks] Describe your approach to designing a new data pipeline from scratch.

1. Understand the business question: what decision does this data enable?
2. Identify sources: what systems own this data? What is the access mechanism?
3. Define the data contract: schema, grain, freshness SLA, quality requirements.
4. Design the transformation logic: medallion architecture layers.
5. Choose tooling: Spark/dbt/SQL based on complexity, team skill, rerun frequency.
6. Build with idempotency from day 1: make reruns safe.
7. Add data quality checks and monitoring before calling it production.
8. Document: data dictionary, lineage, known caveats.

Q152 [Netflix] How do you stay current with the fast-moving data engineering ecosystem?

- Follow: Databricks Blog, Snowflake Blog, Martin Kleppmann's work, Data Engineering Weekly newsletter.
- Community: dbt Slack, Apache Flink/Spark mailing lists, LinkedIn DE community.
- Projects: build personal projects on new tech (Delta Lake, Iceberg, dbt) to learn hands-on.
- Conferences: Data + AI Summit (Databricks), Current (Confluent Kafka), dbt Coalesce.
- Stack Overflow / GitHub: read real-world bug reports and solutions.

Q153 [IBM] What questions would you ask before starting to build a pipeline?

- What business decision or report does this pipeline serve?
- What is the required data freshness (real-time/hourly/daily)?
- What is the acceptable latency SLA?
- Who are the consumers — BI tools, ML models, other pipelines?
- What is the data volume? Growth rate expected?
- What are the data quality requirements? What happens on failure?
- Are there compliance requirements (GDPR, HIPAA, SOC2)?
- What already exists that I can reuse vs build from scratch?

CHAPTER 9 — QUICK-FIRE & MISCELLANEOUS (Questions 331+)

Q154 [General] What is the difference between OLTP and OLAP?

OLTP: Online Transaction Processing. Many short read/write transactions. Row-oriented storage. Normalized schema. E.g. PostgreSQL for an order management system.

OLAP: Online Analytical Processing. Complex aggregation queries on large datasets. Columnar storage. Denormalized star schema. E.g. Snowflake, Redshift for analytics.

Q155 [General] What is columnar storage? Why is it better for analytics?

Columnar storage saves each column's values contiguously on disk (vs row storage which saves each row contiguously).

Analytics advantage: queries often need only a few columns (SELECT SUM(amount) FROM orders). Columnar reads only those columns, not entire rows. Also enables better compression (same-type values compress well). 10-100x fewer I/O bytes for typical analytical queries.

Q156 [IBM] What is Parquet? What are its key advantages?

Parquet: open-source columnar file format. Developed by Twitter/Cloudera.

- Columnar: efficient column-pruning in queries.
- Compressed: Snappy/gzip/Zstd per column. Typically 3-10x smaller than CSV.
- Schema embedded: self-describing. No separate schema file needed.
- Predicate pushdown: Parquet row groups store min/max metadata — Spark skips groups that can't match filter.
- Splittable: can be processed in parallel across tasks.

Q157 [General] Compare Parquet vs Avro vs ORC vs CSV for different use cases.

- Parquet: columnar, analytics/OLAP, Spark default, best for column-heavy queries.
- Avro: row-oriented, excellent for streaming/Kafka (schema evolution via schema registry), row-heavy reads.
- ORC: columnar, optimized for Hive. Good DW format for Hive/Presto workloads.
- CSV: human-readable, no compression, no schema. Only for small data exchange or debugging.

Q158 [Databricks] What is data skewness vs data imbalance?

Data skewness: uneven distribution of values within a column (e.g., 80% of orders belong to 5% of customers). Causes slow joins and group-bys in Spark.

Data imbalance: in ML context — class imbalance in labels (95% "not fraud", 5% "fraud"). Causes ML models to be biased toward majority class.

Q159 [General] What is a surrogate key vs a natural key?

Natural key: a real-world identifier from the source system (e.g., SSN, order_number, email). May change; may not be unique across systems.

Surrogate key: system-generated integer ID with no business meaning (e.g., customer_key IDENTITY). Stable, unique, never changes. Standard in DW dim tables.

Q160 [AWS] What is AWS Glue? When would you use it vs EMR vs Lambda for ETL?

- Glue: managed serverless Spark ETL + data catalog. Good for: scheduled ETL jobs, catalog management, pay-per-DPU-hour (no cluster management).
- EMR: managed Hadoop/Spark clusters. Good for: large-scale jobs needing full cluster control, spot instance optimization, custom configs.
- Lambda: serverless functions. Good for: lightweight event-triggered transforms (<15 min, <10GB). Not for heavy Spark workloads.

Q161 [General] What is the difference between ETL and ELT?

ETL (Extract-Transform-Load): transform data BEFORE loading into warehouse. Traditional approach when DWs were expensive and couldn't run transformations efficiently.

ELT (Extract-Load-Transform): load raw data first, transform in the warehouse using its compute power. Modern approach — cloud DWs are cheap and powerful.

ELT is now standard: load raw → transform with dbt/Spark inside Snowflake/Databricks.

Q162 [Netflix] What is a DAG in the context of data engineering?

DAG = Directed Acyclic Graph. A set of nodes (tasks) connected by directed edges (dependencies) with no cycles.

In Airflow: defines the order of task execution. In Spark: Catalyst builds a DAG of transformations optimized before execution. In dbt: model dependencies form a DAG (run in dependency order).

Acyclic means no task can depend on itself or create a circular dependency.

Q163 [IBM] What is a checkpoint in Spark Structured Streaming?

Checkpoint: Spark writes the current state (consumed offsets, aggregation state, query progress) to durable storage (S3/HDFS) after each micro-batch.

On restart/failure: Spark reads checkpoint and resumes from the last committed offset. Enables exactly-once processing.

```
stream.writeStream
    .format("delta")
    .option("checkpointLocation","s3://bucket/checkpoints/my_stream")
    .start("s3://bucket/delta/output")
```

Q164 [General] Explain the CAP theorem. Give a real database example for each category.

• CP (Consistent + Partition Tolerant): HBase, Zookeeper, MongoDB (w/ strong consistency). May reject requests during network partition.

• AP (Available + Partition Tolerant): Cassandra, DynamoDB, CouchDB. Returns potentially stale data during partition.

• CA (Consistent + Available, no partition tolerance): traditional single-node RDBMS (PostgreSQL, MySQL standalone). Not distributed — partition is not possible by definition.

Q165 [Amazon] What is a data mesh? How does it differ from a centralized data platform?

Data mesh: decentralized ownership where each domain team owns their data products end-to-end (pipelines, quality, SLAs). Central platform provides shared infrastructure (storage, governance, catalog).

Centralized DW: central data engineering team owns all pipelines. Bottleneck at scale. Data mesh solves the bottleneck by distributing ownership.

Key principles: domain ownership, data as a product, self-serve infrastructure, federated governance.

Q166 [IBM] What is the difference between a database view and a materialized view?

View: virtual table. Query is stored, not data. Executed fresh every time it's queried. Always current but potentially slow.

Materialized view: query result stored physically. Pre-computed. Fast to read. May be stale until refreshed.

Use materialized views for expensive aggregations queried frequently where slight staleness is acceptable.

Q167 [Databricks] How do you optimize a dbt project for a large-scale Snowflake DW?

• Use incremental models for large tables — avoid full refresh on every run.

• Use ephemeral materializations for CTEs that are only intermediate steps.

• Cluster keys: add cluster_by config on large Snowflake tables queried on specific columns.

• Analyze query profiles in Snowflake to find slow models.

• Parallelize: dbt runs models in parallel up to thread count (--threads 8).

• Split large models: break monolithic 500-line models into composable smaller ones.

9B — SQL Deep Dive Continued

Q168 [General] What is the difference between TRUNCATE, DELETE, and DROP?

• DELETE: DML. Removes rows matching WHERE. Logged row-by-row. Can be rolled back. Triggers fire.

• TRUNCATE: DDL. Removes ALL rows instantly (deallocates data pages). Minimal logging. Cannot be rolled back in some DBs. No triggers.

• DROP: DDL. Removes the table object entirely (structure + data + indexes).

For emptying large tables, TRUNCATE is orders of magnitude faster than DELETE.

Q169 [IBM] What is a materialized CTE vs a non-materialized CTE?

Non-materialized (default in most DBs): CTE is inlined as a subquery. Optimizer can push predicates through it.

Materialized: CTE is computed once and stored as a temp result. Forces optimizer to treat it as a black box. Useful when CTE is expensive and referenced many times.

```
-- PostgreSQL: force materialization
WITH my_cte AS MATERIALIZED (SELECT ... FROM big_table)
SELECT * FROM my_cte WHERE x > 1;
```

Q170 [Snowflake] What is a lateral flatten in Snowflake? Give an example with a JSON array.

LATERAL FLATTEN explodes a semi-structured array column into rows.

```
SELECT u.user_id, f.value:tag::STRING AS tag
FROM users u,
  LATERAL FLATTEN(input => u.tags_array) f;
```

-- Equivalent to EXPLODE in Spark. Each element in tags_array becomes a row.

Q171 [Amazon] Explain EXPLAIN ANALYZE output. What do actual vs estimated rows tell you?

EXPLAIN ANALYZE runs the query and shows actual vs estimated row counts at each node.

Actual rows >> estimated rows: stats are stale. Run ANALYZE to update statistics.

Actual rows << estimated rows: query plan allocated too many resources to this step; may have over-allocated memory.

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT ...
```

-- Look for: Seq Scan on large table (add index), Hash Join with high memory, Nested Loop (bad on large tables).

Q172 [Netflix] Write a query to find users who visited 3 or more pages in a single session.

```
SELECT user_id, session_id, COUNT(*) page_views
FROM page_events
GROUP BY user_id, session_id
HAVING COUNT(*) >= 3;
```

Q173 [Stripe] How do you calculate a cumulative distribution (CDF) of transaction amounts?

```
SELECT amount,
  PERCENT_RANK() OVER (ORDER BY amount) AS percentile_rank,
  CUME_DIST()    OVER (ORDER BY amount) AS cdf
FROM transactions;
```

-- CUME_DIST: fraction of rows <= current row. Range [1/n, 1].

-- PERCENT_RANK: (rank-1)/(rows-1). Range [0, 1].

Q174 [IBM] Write a query to detect orphan records (child rows with no parent).

```
-- Orders with no matching customer
SELECT o.order_id, o.customer_id
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Q175 [Meta] How do you efficiently count distinct values across multiple columns simultaneously?

```
SELECT
  COUNT(DISTINCT user_id)    AS unique_users,
  COUNT(DISTINCT session_id) AS unique_sessions,
  COUNT(DISTINCT page_url)   AS unique_pages
FROM page_events;
```

For approximate counts on very large datasets, use HyperLogLog (HLL):

```
SELECT HLL_COUNT.MERGE(HLL_COUNT.INIT(user_id)) FROM events;  -- BigQuery
SELECT APPROX_COUNT_DISTINCT(user_id) FROM events;  -- Snowflake/Redshift
```

Q176 [S&P Global] What is a temporal table? How does it differ from SCD Type 2?

Temporal table (ISO SQL:2011): database-managed system versioning. DB automatically maintains history in a shadow table on every UPDATE/DELETE. Query AS OF SYSTEM TIME syntax.

```
SELECT * FROM instruments FOR SYSTEM_TIME AS OF '2024-01-15';
```

SCD Type 2: manually managed in application layer — need to write the expire/insert logic yourself.

Temporal tables are supported in SQL Server, PostgreSQL (with extension), MariaDB. Simpler but less flexible than custom SCD2.

Q177 [General] What is the difference between a primary key and a unique key?

• Primary Key: uniquely identifies each row. Only ONE per table. Cannot be NULL. Creates clustered index by default (SQL Server).

• Unique Key: enforces uniqueness. Can have MULTIPLE per table. Allows ONE NULL (in most DBs). Creates non-clustered index.

In DW surrogate key pattern: surrogate key = PK, natural key = UK.

Q178 [Netflix] Write a query to find the top 10% of users by watch time.

```
SELECT user_id, total_watch_minutes FROM (
  SELECT user_id, SUM(watch_minutes) total_watch_minutes,
    PERCENT_RANK() OVER (ORDER BY SUM(watch_minutes) DESC) pr
  FROM watch_events GROUP BY user_id
) t WHERE pr <= 0.10;
```

Q179 [Databricks] How do you implement row-level security in Snowflake?

```
-- Create row access policy
CREATE OR REPLACE ROW ACCESS POLICY region_policy
  AS (region_col VARCHAR) RETURNS BOOLEAN ->
  CURRENT_ROLE() IN ('ADMIN', 'SUPERUSER')
  OR region_col = CURRENT_USER();

-- Attach to table
ALTER TABLE sales ADD ROW ACCESS POLICY region_policy ON (region);
```

-- Analysts now only see rows where region = their username (or if they are admin).

Q180 [IBM] Explain what a hash join is and when it is chosen by the optimizer.

Hash join: build a hash table from the smaller relation on the join key, then probe the hash table for each row of the larger relation.

Chosen when: no index on join columns, large tables, equi-joins (=). Not suitable for range/inequality joins.

Two phases: Build (hash smaller table into memory) → Probe (scan larger table, hash-lookup). If build table does not fit in memory → grace hash join (partition to disk first).

Q181 [Amazon] What is a nested loop join? When is it efficient?

Nested loop: for each row in the outer table, scan the inner table for matches.

O(N*M) — terrible for large tables. But efficient when:

• Outer table is very small (few rows)

• Inner table has an index on the join column (reduces inner scan to O(log N))

• In Spark: only used as broadcast join internally (broadcast = send small table to all nodes)

Q182 [Teradata] What is SAMPLE in Teradata? How does it differ from PERCENT?

```
-- SAMPLE: return approximately N random rows
SELECT * FROM large_table SAMPLE 1000;

-- SAMPLE PERCENT: return approximately N% of rows
SELECT * FROM large_table SAMPLE 5 PERCENT;
```

Teradata SAMPLE is non-deterministic (random AMP-level selection). Useful for quick data profiling on massive tables without full scans.

Q183 [General] What is a foreign key constraint? Should you use them in a data warehouse?

FK constraint: enforces referential integrity — a column must reference a valid PK in the parent table. DB rejects invalid inserts.

In OLTP: yes, use them for data integrity.

In DW (Snowflake, Redshift, BigQuery): typically declare FK relationships as metadata only (not enforced). Loading speed and flexibility matter more. Enforce integrity via ETL logic and dbt tests instead.

```
-- Snowflake: non-enforced FK (just metadata for lineage tools)
ALTER TABLE fact_orders ADD FOREIGN KEY (customer_key) REFERENCES
dim_customer(customer_key) NOT ENFORCED;
```

Q184 [Airbnb] Write a query to compute a 30-day retention rate from signup cohorts.

```
WITH cohort AS (
```

```
  SELECT user_id, DATE_TRUNC('day', signup_date) cohort_day FROM users
),
retained AS (
  SELECT DISTINCT user_id FROM events
  WHERE event_date BETWEEN signup_date+29 AND signup_date+31
)
SELECT c.cohort_day,
  COUNT(c.user_id)  cohort_size,
  COUNT(r.user_id)  retained_30d,
  ROUND(100.0*COUNT(r.user_id)/COUNT(c.user_id),1) retention_pct
FROM cohort c LEFT JOIN retained r ON c.user_id=r.user_id
GROUP BY c.cohort_day ORDER BY c.cohort_day;
```

Q185 [General] What is an index scan vs a full table scan? Which is always better?

Full table scan: read every row sequentially. Good for: large result sets (>10-20% of table), small tables, no useful index.

Index scan: traverse index structure to find matching rows, then fetch from table. Good for: selective queries returning few rows (<5% of table).

Index scan is NOT always better. For a query returning 80% of rows, full table scan is faster — sequential I/O outperforms random I/O from constant index lookups.

Q186 [Meta] Write a query to find the most common value (mode) per group.

```
SELECT category, product FROM (
  SELECT category, product,
    ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(*) DESC) rn
  FROM orders
  GROUP BY category, product
) t WHERE rn = 1;
```

Q187 [IBM] How would you compare two tables to find rows that differ?

```
-- Rows in A not in B (by full row hash)
(SELECT * FROM table_a EXCEPT SELECT * FROM table_b)
UNION ALL
(SELECT * FROM table_b EXCEPT SELECT * FROM table_a);

-- Or hash-based comparison
SELECT a.id,
  MD5(CONCAT(a.col1,a.col2)) a_hash,
  MD5(CONCAT(b.col1,b.col2)) b_hash
FROM table_a a JOIN table_b b ON a.id=b.id
WHERE MD5(CONCAT(a.col1,a.col2)) != MD5(CONCAT(b.col1,b.col2));
```

9C — PySpark Advanced Questions

Q188 [Databricks] What is a DataFrame vs Dataset vs RDD in Spark? Which should you use?

- RDD: low-level distributed collection. No schema, no optimization. Use only when needing fine-grained control.
- Dataset: typed API (Scala/Java). Schema + compile-time type safety. Not available in Python.
- DataFrame: distributed table with schema. Catalyst-optimized. Python/Scala/Java/R. Default choice.

Always use DataFrame API in PySpark. RDD only when DataFrame cannot express the operation.

Q189 [Netflix] How do you read from Kafka in PySpark Structured Streaming?

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StringType, LongType

schema = StructType().add("user_id",StringType()).add("amount",LongType())

stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers","kafka:9092") \
    .option("subscribe","transactions") \
```

```
    .option("startingOffsets","latest") \
    .load() \
    .select(F.from_json(F.col("value").cast("string"),schema).alias("d")) \
    .select("d.*")
```

Q190 [IBM] Explain Spark broadcast variables and accumulators.

Broadcast variable: read-only data sent to all executors ONCE (not per task). Use for lookup tables/configs shared across tasks.

```
lookup = spark.sparkContext.broadcast({"USD":1.0,"EUR":0.92})
@F.udf(returnType=DoubleType())
def convert(amount, currency): return amount * lookup.value.get(currency, 1.0)
```

Accumulator: write-only counter/sum from executors, readable on driver. Use for counting bad records.

```
bad_records = spark.sparkContext.accumulator(0)
def parse(row):
    if row.amount < 0: bad_records.add(1)
    return row
df.foreach(parse)
print(f"Bad records: {bad_records.value}")
```

Q191 [Amazon] How do you write a PySpark job that processes files incrementally (only new files)?

```
# Option 1: Structured Streaming with maxFilesPerTrigger
stream = spark.readStream \
    .format("parquet") \
    .option("maxFilesPerTrigger",10) \
    .schema(schema) \
    .load("s3://bucket/raw/")

# Option 2: Track processed files in a Delta table
processed = spark.read.format("delta").load("s3://meta/processed_files")
all_files = spark.createDataFrame(dbutils.fs.ls("s3://bucket/raw/"))
new_files = all_files.join(processed,"path","left_anti")
```

Q192 [Databricks] How do you handle corrupt or malformed records in PySpark?

```
# PERMISSIVE (default): parse good records, put bad ones in _corrupt_record column
df = spark.read.option("mode","PERMISSIVE") \
    .option("columnNameOfCorruptRecord","_corrupt") \
    .json("s3://bucket/events/")

# DROPMALFORMED: silently drop bad records
df = spark.read.option("mode","DROPMALFORMED").json("s3://...")

# FAILFAST: throw exception on first bad record
df = spark.read.option("mode","FAILFAST").json("s3://...")

# Best practice: PERMISSIVE + quarantine corrupt records
good = df.filter(F.col("_corrupt").isNull())
bad  = df.filter(F.col("_corrupt").isNotNull())
bad.write.mode("append").json("s3://quarantine/")
```

Q193 [Netflix] What is Spark speculative execution? Should you always enable it?

Speculative execution: Spark launches duplicate copies of slow tasks on other executors. First to finish wins; others killed.

Enable for: production jobs with SLA sensitivity where one straggler task can delay the whole job.

Disable when: tasks have side effects (writing to external DB without idempotency), or tasks are slow due to data skew (launching duplicates wastes resources without fixing skew).

```
spark.conf.set("spark.speculation","true")
spark.conf.set("spark.speculation.multiplier","1.5")  # task taking 1.5x median is a
straggler
```

Q194 [IBM] How do you convert a wide DataFrame to long format in PySpark?

```
from pyspark.sql.functions import array, struct, col, explode, lit
```

```
# Wide: user_id, q1_sales, q2_sales, q3_sales
# Long: user_id, quarter, sales

df_long = df.select(
    "user_id",
    explode(array(
        struct(lit("Q1").alias("quarter"), col("q1_sales").alias("sales")),
        struct(lit("Q2").alias("quarter"), col("q2_sales").alias("sales")),
        struct(lit("Q3").alias("quarter"), col("q3_sales").alias("sales")),
    )).alias("kv")
).select("user_id","kv.quarter","kv.sales")
```

Q195 [Amazon] How do you optimize a Spark job that reads millions of small JSON files from S3?

1. Use wholeTextFiles or sc.binaryFiles to group small files before parsing.
2. Pre-compact small files to Parquet using a one-time batch job (coalesce + write).
3. Use S3 Select to push predicate to S3 level (reduces bytes transferred).
4. Increase spark.sql.files.maxPartitionBytes to merge small files into fewer tasks.

```
spark.conf.set("spark.sql.files.maxPartitionBytes", str(256*1024*1024))  # 256MB
spark.conf.set("spark.sql.files.openCostInBytes", str(8*1024*1024))     # 8MB open cost
```

Q196 [Databricks] What is the difference between mapPartitions and map in Spark?

map: applies function to each row individually. Per-row overhead (DB connections opened per row).

mapPartitions: applies function to an iterator of all rows in a partition. One function call per partition. Ideal for batching DB writes, loading ML models once per partition.

```
# Load model once per partition, not per row
def predict_partition(rows):
    model = load_model()  # loaded once
    for row in rows:
        yield (row.id, model.predict(row.features))

result = df.rdd.mapPartitions(predict_partition)
```

Q197 [Netflix] How do you efficiently join a 10TB table with a 50GB table in Spark?

50GB is above default broadcast threshold (10MB) but small enough for a tuned broadcast with enough executor memory.

```
# Option 1: Increase broadcast threshold if executors have enough RAM
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 60*1024*1024*1024)  # 60GB (needs
~60GB RAM per executor)

# Option 2: Bucket both tables on join key (eliminates shuffle)
large_df.write.bucketBy(256,"user_id").saveAsTable("large_bucketed")
medium_df.write.bucketBy(256,"user_id").saveAsTable("medium_bucketed")
spark.table("large_bucketed").join(spark.table("medium_bucketed"),"user_id")

# Option 3: Repartition medium on join key before join to reduce shuffle size
medium_repartitioned = medium_df.repartition(256,"user_id")
```

9D — Data Modeling Questions

Q198 [IBM] What is the Kimball methodology? Key principles.

Kimball = dimensional modeling for data warehouses. Key principles:

- Business process focus: model facts around measurable business events.
- Grain declaration: state explicitly what one row represents.
- Dimensions for context: who, what, where, when, why, how.
- Conformed dimensions: shared dims across fact tables enable drill-across queries.
- Surrogate keys: system-generated keys, never natural keys, in dims.
- Bus Matrix: documents which dims apply to which facts.

Q199 [Amazon] What is a snowflake schema? When would you use it over star schema?

Snowflake schema: normalized dimension tables (dims reference other dims). Reduces storage redundancy.

Star schema: denormalized dims (all attributes in one dim table). Simpler queries, faster reads.

Use snowflake when: storage is extremely constrained, or dimension hierarchy is deep and shared across multiple dims.

Use star schema when: query performance and simplicity matter (BI tools work better with star). This is the default recommendation.

Q200 [Databricks] What is a Data Vault 2.0? When would you use it over Kimball?

Data Vault 2.0: hub-link-satellite pattern for modeling enterprise data with full auditability.

• Hub: business key only (e.g., customer_id). Hash key as surrogate.

• Link: relationship between hubs (e.g., order connecting customer + product hubs).

• Satellite: descriptive attributes + history (loads over time).

Use Data Vault when: regulatory auditability required, multiple source systems with conflicting data, need full history from day 1, agile schema evolution expected.

Use Kimball for: simpler BI-focused DWs where flexibility > auditability.

Q201 [Netflix] What is the difference between additive, semi-additive, and non-additive metrics?

• Additive: can be summed across ALL dimensions. Revenue, order count, clicks. SUM(revenue) across time + region + product is always valid.

• Semi-additive: can sum across SOME but not all dimensions. Bank account balance — sum across accounts is fine, but sum across time is meaningless (balance on Jan 31 ≠ Jan balance + Feb balance).

• Non-additive: cannot meaningfully sum across any dimension. Ratios, percentages, conversion rates. AVG or ratio calculation required.

Q202 [IBM] Design a fact table for a ride-sharing company (Uber/Lyft). What is the grain?

Grain: one row per ride.

```
CREATE TABLE fact_rides (
  ride_key         BIGINT,       -- surrogate key
  driver_key       INT,          -- FK dim_driver
  rider_key        INT,          -- FK dim_rider
  pickup_date_key  INT,          -- FK dim_date
  pickup_loc_key   INT,          -- FK dim_location
  dropoff_loc_key  INT,          -- FK dim_location
  vehicle_key      INT,          -- FK dim_vehicle
  ride_duration_min  DECIMAL,    -- additive measure
  ride_distance_km   DECIMAL,    -- additive measure
  fare_amount        DECIMAL,    -- additive measure
  surge_multiplier   DECIMAL,    -- non-additive (use AVG)
  driver_rating      DECIMAL     -- semi-additive
);
```

Q203 [Snowflake] What is a junk dimension? Give an example.

Junk dimension: a dim that combines multiple low-cardinality flags/indicators into a single dim table, avoiding many small flag columns in the fact table.

```
-- Instead of 5 flag columns in fact table:
-- is_promo_purchase, is_first_order, is_mobile, is_gift, is_international

-- Create junk dim with all combinations:
CREATE TABLE dim_order_flags (
  flag_key          INT PRIMARY KEY,
  is_promo_purchase BOOLEAN,
  is_first_order    BOOLEAN,
  is_mobile         BOOLEAN,
  is_gift           BOOLEAN,
  is_international  BOOLEAN
);
```

-- Fact table references flag_key. Keeps fact table narrow.

Q204 [IBM] What is a role-playing dimension?

A single physical dim table used in multiple roles within the same fact table.

Example: dim_date used 3 times in fact_orders: as order_date_key, ship_date_key, delivery_date_key.

Implementation: create SQL views or aliases for each role.

```
CREATE VIEW dim_order_date   AS SELECT * FROM dim_date;
CREATE VIEW dim_ship_date    AS SELECT * FROM dim_date;
CREATE VIEW dim_delivery_date AS SELECT * FROM dim_date;
```

-- BI tool treats each view as a separate dimension with its own relationship to the fact.

Q205 [Teradata] What is a late-arriving dimension in DW? How do you handle it?

Late-arriving dim: a dimension record arrives AFTER the fact row that references it.

Example: a new product shipped today appears in sales facts, but the product master record is not yet in dim_product.

Handling options:

1. Create a placeholder "Unknown Product" dim row with surrogate key -1. Update fact to proper key when dim arrives.

2. Hold fact rows in a staging/quarantine table until dim record arrives, then load both.

3. Use a default "Infer member" surrogate key and reconcile in overnight batch.

9E — Cloud, DevOps & Infrastructure

Q206 [AWS] Compare S3 storage classes for data lake cost optimization.

- S3 Standard: frequently accessed data. Highest cost. Use for hot/active data (Bronze recent, Gold layer).
- S3 Standard-IA (Infrequent Access): 40% cheaper storage, retrieval fee. Use for Silver/older Bronze.
- S3 Glacier Instant Retrieval: archive with ms retrieval. Use for data accessed <quarterly.
- S3 Glacier Deep Archive: cheapest storage ($0.00099/GB). 12-48hr retrieval. Use for regulatory/compliance archives.
- S3 Intelligent-Tiering: auto-moves objects between tiers based on access patterns. Good for unpredictable access.

Tip: use S3 Lifecycle policies to auto-transition: Standard → IA after 30 days → Glacier after 90 days.

Q207 [General] What is Infrastructure as Code? What IaC tools do data engineers use?

IaC: define infrastructure (clusters, buckets, queues, IAM roles) in version-controlled code files. Reproducible, auditable, automated.

- Terraform: cloud-agnostic HCL. Most widely used. Manages AWS/GCP/Azure/Databricks/Snowflake resources.
- AWS CloudFormation: AWS-native JSON/YAML. Tight AWS integration.
- Pulumi: IaC in Python/TypeScript — appeals to developers.
- Databricks Asset Bundles (DAB): Databricks-native IaC for jobs, clusters, pipelines.

Q208 [IBM] Explain CI/CD for data pipelines. What does a data pipeline CI/CD pipeline look like?

CI (Continuous Integration): on every PR — run linters (sqlfluff, black), unit tests (pytest, dbt test), schema validation.

CD (Continuous Deployment): on merge to main — deploy dbt models to staging → run integration tests → promote to production.

```
# GitHub Actions example
on: [push]
jobs:
  test:
    steps:
      - run: pip install dbt-snowflake
      - run: dbt compile --profiles-dir .
      - run: dbt test --profiles-dir . --target dev
      - run: pytest tests/
```

Q209 [Netflix] What is Docker and why do data engineers use it?

Docker: containerization platform. Packages application + dependencies into a portable image.

Data engineering uses: containerize Spark jobs (consistent environment), run Airflow locally (docker-compose), deploy dbt in containers, ensure dev/staging/prod parity.

```
# Dockerfile for a PySpark job
FROM python:3.10-slim
RUN pip install pyspark==3.5.0 delta-spark==3.0.0
COPY . /app
WORKDIR /app
ENTRYPOINT ["python","main.py"]
```

Q210 [Amazon] What is Kubernetes? How does it relate to data engineering workloads?

Kubernetes (K8s): container orchestration system. Manages deployment, scaling, and lifecycle of containerized applications.

Data engineering uses: run Airflow on K8s (KubernetesExecutor — each task is a pod), deploy Spark on K8s (spark-on-k8s operator), run MLflow/FastAPI serving.

KubernetesExecutor advantage: each Airflow task spins up a fresh pod with its own resources. No shared worker contention. Perfect isolation.

Q211 [Databricks] What is the difference between Databricks Jobs, Workflows, and DLT pipelines?

- Job: run a single notebook or script on a schedule or trigger. Simplest.
- Workflow: multi-task job with dependencies (DAG of tasks). Replaces simple Airflow for Databricks-native pipelines.
- Delta Live Tables (DLT): declarative pipeline framework. Define tables; DLT handles scheduling, retries, DQ checks, lineage. Best for streaming + incremental batch pipelines.

Use Workflows for ad-hoc multi-step jobs. Use DLT for production data pipelines with quality requirements.

Q212 [IBM] What is data observability? Name the 5 pillars.

Data observability: continuous visibility into the health, reliability, and quality of your data and pipelines.

5 Pillars (Monte Carlo framework):

1. Freshness: is data up to date? Did the pipeline run on schedule?
2. Volume: did the expected amount of data arrive? Anomaly = missing/extra data.
3. Schema: did column names, types, or structure change unexpectedly?
4. Distribution: are column values within expected ranges and distributions?
5. Lineage: which upstream sources/transformations produced this data?

Tools: Monte Carlo, Bigeye, Acceldata, Great Expectations, custom dashboards.

Q213 [Netflix] How do you manage secrets (passwords, API keys) in a data pipeline?

NEVER hardcode secrets in code or config files committed to version control.

- AWS Secrets Manager / Parameter Store: store secrets, retrieve via SDK at runtime.
- HashiCorp Vault: centralized secrets management, dynamic credentials.
- Airflow Connections: encrypted secrets in Airflow metadata DB, referenced by conn_id.
- Databricks Secrets: dbutils.secrets.get(scope, key).
- Environment variables: set at container/cluster launch, not in code.

```
# Airflow: reference secret by conn_id
hook = SnowflakeHook(snowflake_conn_id="snowflake_prod")
```

Q214 [Amazon] What is schema registry? Why is it important for Kafka pipelines?

Schema Registry (e.g., Confluent Schema Registry): centralized service that stores and validates message schemas (Avro/Protobuf/JSON Schema) for Kafka topics.

Why important:

- Schema enforcement: producers cannot send messages with invalid schemas.
- Schema evolution: controlled changes with forward/backward compatibility checks.
- Consumer safety: consumers always know what schema to expect.
- Compact wire format: Avro with schema ID is far smaller than JSON with field names.

Q215 [Databricks] What is Apache Iceberg and how does it compare to Delta Lake?

- Both: open table formats on Parquet providing ACID, schema evolution, time travel, hidden partitioning.
- Delta Lake: Databricks-native. Deep Spark integration. Delta Sharing. Best in Databricks ecosystem.

• Iceberg: Apache project. Engine-agnostic (Flink, Trino, Hive, Dremio, Spark). Better partition evolution (can change partition spec without rewrite). Row-level deletes via position/equality delete files.

• Hudi: yet another OTF. Specialized for upsert-heavy workloads (Uber's use case).

Multi-engine shop → Iceberg. Databricks shop → Delta Lake.

9F — Data Engineering Concepts & Theory

Q216 [General] What is eventual consistency? Give a data engineering example.

Eventual consistency: in a distributed system, replicas will converge to the same state eventually, but at any point in time they may be out of sync.

Example: Cassandra with RF=3 and QUORUM reads. If you write to one replica and immediately read from another before replication completes, you may see stale data. After a few milliseconds, all replicas converge.

In data pipelines: a Gold layer metric updated at 6 AM may not reflect events that arrived at 5:59 AM — they'll be included in the next run (eventual consistency at pipeline level).

Q217 [IBM] What is a Bloom filter? How is it used in Parquet/Delta?

Bloom filter: probabilistic data structure that answers "is this value definitely NOT in the set?" with zero false negatives but possible false positives.

In Parquet/Delta: stored per column per row group. When querying WHERE user_id = "XYZ", Spark checks the bloom filter — if it says "not present," the entire row group is skipped without reading it.

```
# Delta Lake: enable bloom filter on a column
spark.sql("""
  ALTER TABLE events
  SET TBLPROPERTIES ('delta.bloomFilter.user_id.enabled'='true',
                     'delta.bloomFilter.user_id.fpp'='0.1')
""")
```

Q218 [General] What is the difference between horizontal and vertical scaling?

Vertical scaling (scale up): add more CPU/RAM/disk to existing machine. Simpler but has a ceiling (max machine size) and single point of failure.

Horizontal scaling (scale out): add more machines. Distributed. No hard ceiling. Requires distributed system design.

In data engineering: Spark/Kafka/Cassandra are horizontally scalable. A single large PostgreSQL server is vertical. Cloud DWs (Snowflake, BigQuery) scale both compute and storage independently.

Q219 [Netflix] What is the difference between push and pull data ingestion patterns?

Pull: DE pipeline queries/polls the source system on a schedule. Simple. Source does not need to know about the pipeline. Risk: source load at poll time; may miss events between polls.

Push: source system pushes events to pipeline (webhooks, Kafka producers) as they happen. Lower latency. Source must be modified to push.

Modern pattern: push to Kafka (event streaming) → pipeline consumes at its own pace. Decouples producer and consumer speed.

Q220 [IBM] What is a distributed transaction? Why is it hard in distributed systems?

Distributed transaction: atomic operation spanning multiple nodes/services. All succeed or all fail.

Hard because: 2-Phase Commit (2PC) coordinator is a bottleneck and single point of failure; network partitions can leave participants in uncertain state; high latency due to coordination rounds.

Modern alternatives: Saga pattern (sequence of local transactions with compensating actions); eventual consistency + idempotent retries; Kafka transactions for producer-side atomicity.

Q221 [Amazon] What is a data lake vs a data lakehouse?

Data lake: raw data dumped to object storage (S3/ADLS). Cheap, flexible, any format. But: no ACID, no schema enforcement, no performance optimization.

Lakehouse: data lake + warehouse capabilities. ACID transactions, schema evolution, time travel, query optimization — on top of open formats (Delta Lake, Iceberg, Hudi).

Lakehouse eliminates the need for a separate DW tier. One platform for both raw data and curated analytics.

Q222 [General] What is the two-phase commit protocol? Explain in context of Kafka.

2PC: distributed atomicity protocol.

Phase 1 (Prepare): coordinator asks all participants if they are ready to commit. Each votes yes/no.

Phase 2 (Commit/Abort): if all vote yes, coordinator sends commit. If any votes no, coordinator sends abort.

Kafka transactions use a variant: transactional producer sends all messages + commits offset atomically. Consumers with isolation.level=read_committed only see fully committed batches.

Q223 [Databricks] What is a surrogate key and how do you generate them at scale in Spark?

```
# Option 1: monotonically_increasing_id (unique but not sequential)
df.withColumn("surrogate_key", F.monotonically_increasing_id())

# Option 2: Hash-based (deterministic, reproducible)
df.withColumn("sk", F.sha2(F.concat_ws("||","source_id",F.lit("customers")),256))

# Option 3: zipWithIndex on RDD (truly sequential, but requires action)
rdd = df.rdd.zipWithIndex()
```

-- monotonically_increasing_id is the most common. Note: gaps exist across partitions.

Q224 [Netflix] What is data freshness? How do you measure and monitor it?

Data freshness: how up-to-date the data is relative to the real-world events it represents.

Measure: MAX(updated_at) or MAX(event_date) in the table. Compare against current time.

```
-- Freshness check in dbt
sources:
  - name: raw
    tables:
      - name: orders
        freshness:
          warn_after: {count: 1, period: hour}
          error_after: {count: 3, period: hour}
        loaded_at_field: updated_at
```

Monitor: Airflow SLA miss callbacks, custom SQL freshness check tasks, Monte Carlo data observability.

Q225 [IBM] Explain the concept of a hot partition in distributed systems. How do you avoid it?

Hot partition: one partition receives disproportionately more reads/writes than others. One node becomes a bottleneck while others are idle.

Causes: sequential keys (all writes go to last partition), popular entity (one celebrity's data in Cassandra).

Fixes:

- Random UUID as partition key to distribute writes.
- Add a random bucket suffix to hot keys (salting).
- Cassandra: use composite partition key (user_id, bucket) where bucket = HASH(user_id) % 10.
- Kafka: choose high-cardinality partition keys; avoid null keys (all go to same partition).

Q226 [Amazon] What is columnar compression and how does it work in Parquet?

Columnar compression: compress each column independently. Because columns contain values of the same type and often similar values, compression ratios are much higher than row-based compression.

Parquet compression algorithms:

- Snappy: fast compress/decompress, moderate ratio. Default in Spark.
- Gzip: higher ratio, slower. Good for archival.
- Zstd: best balance of ratio + speed. Recommended for production.
- Dictionary encoding: map repeated column values to small integer codes (e.g., 1000 rows with "CONFIRMED" stored as code 1).
- RLE (Run Length Encoding): consecutive identical values stored as (value, count) pairs.

9G — Teradata & IBM-Specific Advanced

Q227 [Teradata] What is a Partition Primary Index (PPI) in Teradata?

PPI combines Teradata's AMP distribution (PI) with partition elimination (like date-based partitioning).

Rows are distributed to AMPs by PI hash, then within each AMP stored in partitions.

```
CREATE TABLE orders (
  order_id INT, customer_id INT, order_date DATE, amount DECIMAL(18,2)
)
PRIMARY INDEX (customer_id)
PARTITION BY RANGE_N(order_date BETWEEN DATE '2020-01-01' AND DATE '2025-12-31'
  EACH INTERVAL '1' MONTH);
```

Queries filtering on order_date AND customer_id use both partition elimination AND PI lookup.

Q228 [Teradata] What is VOLATILE TABLE in Teradata? When do you use it?

Volatile table: session-scoped temporary table. Automatically dropped at session end. Stored in spool space (not permanent space). Not logged.

```
CREATE VOLATILE TABLE tmp_active_customers AS (
  SELECT customer_id FROM customers WHERE status = 'active'
) WITH DATA PRIMARY INDEX (customer_id) ON COMMIT PRESERVE ROWS;
```

Use for: intermediate results in complex multi-step queries, avoiding repeated expensive subqueries within a session.

Q229 [Teradata] What is BTEQ scripting? Give a BTEQ script structure.

```
.LOGON server/username,password;

.IF ERRORCODE <> 0 THEN .QUIT 8;

-- SQL commands
DELETE FROM staging_orders;

.IF ERRORCODE <> 0 THEN .GOTO ERRHANDLE;

INSERT INTO staging_orders SELECT * FROM raw_orders WHERE load_date = '2024-01-15';

.LABEL ERRHANDLE
.IF ERRORCODE <> 0 THEN .QUIT 8;

.LOGOFF;
.QUIT 0;
```

Q230 [IBM] What is Db2 pureScale? How does it differ from standard Db2?

Db2 pureScale: shared-disk cluster architecture for high availability and horizontal scalability. Multiple database members share a single storage layer (GPFS).

Standard Db2: single-node or simple HA (primary/standby). No shared-disk cluster.

pureScale advantage: add members (nodes) without application downtime; all members see the same data; member failure does not impact other members.

Used for: mission-critical OLTP workloads requiring 99.999% availability.

Q231 [Teradata] What is the difference between INNER and CROSS joins in Teradata?

Same semantics as standard SQL. Teradata-specific note: without a WHERE join condition, Teradata will produce a product join (CROSS JOIN) which can be massive.

```
-- Accidental cross join (missing join condition):
SELECT * FROM orders, customers;  -- CROSS JOIN! orders * customers rows

-- Teradata optimizer warning: "Product Join" in EXPLAIN output
-- Always use explicit JOIN ... ON syntax to avoid this.
```

Q232 [IBM] What is Hadoop? Is it still relevant in 2025?

Hadoop: open-source framework for distributed storage (HDFS) and processing (MapReduce).

In 2025: mostly legacy. Key components still relevant:

- HDFS: largely replaced by cloud object storage (S3, GCS, ADLS). Still used in on-prem.
- YARN: resource manager. Still used to run Spark in many on-prem clusters.
- Hive Metastore: the catalog/metadata layer is still widely used (even by Spark, Databricks, Trino).

• MapReduce: effectively dead. Replaced by Spark (10-100x faster).

Modern stacks: Spark + S3 + Delta Lake replaced the full Hadoop ecosystem for new projects.

9H — Streaming & Real-time Advanced

Q233 [General] What is Apache Flink? How does it differ from Spark Structured Streaming?

• Flink: true stream processor. Row-by-row event processing. Sub-second latency. Stateful with RocksDB. Designed for streaming first.

• Spark Structured Streaming: micro-batch (default). Easier for Spark teams. Slightly higher latency (seconds). Better for teams already on Spark.

Flink wins: latency-critical use cases (fraud detection, real-time recommendations, <100ms SLA).

Spark wins: teams already on Databricks/Spark, need tight integration with batch jobs, latency > 1 second is OK.

Q234 [Netflix] What is stateful streaming? Give a practical example.

Stateful streaming: processing that maintains state across events (not just processing each event independently).

Example: count unique users per session. Requires tracking which users have been seen in the current session window — that's state.

```
# Spark: stateful aggregation with watermark
stream.withWatermark("event_time","30 minutes")
    .groupBy("session_id")
    .agg(F.count_distinct("user_id").alias("unique_users"))
```

State is stored in executor memory (with RocksDB backend for large state). Checkpointed to durable storage for fault tolerance.

Q235 [IBM] What is a tumbling window vs sliding window vs session window?

• Tumbling: fixed-size, non-overlapping. Every event belongs to exactly one window. E.g., 1-hour windows: [0-1h], [1-2h], [2-3h].

• Sliding: fixed-size, overlapping. Events can belong to multiple windows. E.g., 1-hour window every 15 min: [0-1h], [0:15-1:15h]...

• Session: gap-based. Window closes after N minutes of inactivity. Size varies. E.g., session ends after 30 min idle.

```
# Spark tumbling window
F.window("event_time","1 hour")
# Spark sliding window
F.window("event_time","1 hour","15 minutes")
```

Q236 [Amazon] What is Kafka Streams vs Kafka consumer + Spark Streaming?

Kafka Streams: Java library for stream processing directly in Kafka. No separate cluster needed. State stored in Kafka topics. Good for simple stateful transformations close to Kafka.

Spark Structured Streaming: separate compute cluster. Full Spark SQL support, DataFrame API, complex ML integration. Better for heavy transformations, joins with large datasets.

Choose Kafka Streams for: lightweight, low-latency, microservice-embedded processing. Choose Spark for: complex transformations, large-scale joins, ML integration.

Q237 [Netflix] How do you implement deduplication in a Kafka-to-Delta streaming pipeline?

```
def write_deduped(df, epoch_id):
    from delta.tables import DeltaTable
    # Dedup within micro-batch first
    deduped = df.dropDuplicates(["event_id"])
    # MERGE into Delta (upsert = idempotent)
    target = DeltaTable.forPath(spark,"s3://delta/events")
    target.alias("t").merge(
        deduped.alias("s"),
        "t.event_id = s.event_id"
    ).whenNotMatchedInsertAll().execute()

stream.writeStream.foreachBatch(write_deduped)
    .option("checkpointLocation","s3://ckpt/events").start()
```

Q238 [Databricks] What is a Kafka topic replication factor and why does it matter?

Replication factor (RF): number of copies of each partition stored across different brokers.

RF=1: single copy. If broker dies, data lost permanently.

RF=2: one replica. Can survive 1 broker failure.

RF=3: two replicas. Standard production setting. Can survive 2 broker failures. Recommended minimum.

min.insync.replicas: minimum replicas that must acknowledge a write before producer considers it committed. Typically set to RF-1.

```
# Topic creation with RF=3
kafka-topics.sh --create --topic events --replication-factor 3 --partitions 24
```

9I — More System Design Scenarios

Q239 [Amazon] Design a pipeline to detect anomalies in IoT sensor data in real time.

Ingest: IoT devices → MQTT broker → Kafka (topic per device class). 1M+ devices, millions of events/min.

Stream: Flink consumes Kafka. Compute rolling stats (mean, stddev) per device per window.

Anomaly detection: Z-score > 3σ → anomaly flag. Or ML model scoring via Flink async I/O.

Alert: anomaly events → alert Kafka topic → notification service (PagerDuty/SNS).

Store: all events → S3/Delta (Bronze). Anomaly events → DynamoDB (queryable by device+time).

Replay: if algorithm improves, replay Kafka history to re-score historical events.

Q240 [Netflix] Design a data pipeline for AB testing result analysis.

Capture: experiment assignment events + outcome events (conversion, watch time) → Kafka.

Join: Flink or Spark joins experiment assignment with outcome events by user_id within experiment window.

Compute: per-variant metrics (conversion rate, average watch time, revenue). Statistical significance (t-test, p-value).

Store: experiment results → Snowflake/Redshift experiment_results table.

Serve: experimentation dashboard (Looker/internal) queries results. Shows variants, sample sizes, lift, confidence intervals.

Guard: automated guardrail metrics (latency, error rate) stop bad experiments automatically.

Q241 [IBM] Design a GDPR-compliant data retention and deletion system.

Data inventory: tag all tables/columns with data classification (PII, sensitive, public) in Glue/Purview.

Retention policy: configure per-data-class retention rules. After retention period → auto-delete via Airflow DAG.

```
-- Mark expired records
DELETE FROM orders WHERE created_at < CURRENT_DATE - INTERVAL 7 YEAR;
```

Right to deletion: on request, delete or anonymize all records for a user_id across all systems. Log deletion in audit table.

Pseudonymization: replace PII with hash at ingest. Delete the hash→PII mapping to effectively anonymize.

Audit: immutable audit log of all data access and deletion events.

Q242 [Databricks] Design a data quality monitoring system for a large data platform.

Rule layer: define rules per table: not_null, unique, range, referential integrity, custom SQL.

Execution: run checks after each pipeline run (Airflow task). Log results to a central quality_checks table.

Alerting: if any check fails → Slack/PagerDuty alert with table name, check description, failure count.

Dashboard: Grafana/Superset dashboard showing DQ pass rate over time per table.

Anomaly detection: volume and distribution drift detection beyond static rules.

Quarantine: failed records routed to quarantine zone; pipeline proceeds with good records; quarantine reviewed daily.

Q243 [S&P Global] Design a multi-tenant data platform serving 50 enterprise clients.

Isolation: each client gets dedicated schema (Snowflake) or separate Delta table path with prefix.

Access control: RBAC — each client's users can only access their schema. Row access policies for shared reference data.

Compute isolation: separate virtual warehouses per client in Snowflake, or separate clusters in Databricks (workspaces).

Onboarding: templated IaC (Terraform) to provision new client environment in minutes.

Billing: track credit/compute usage per client schema/warehouse for chargeback.

Data quality: per-client DQ checks. Failures in one client's pipeline don't affect others.

Governance: centralized catalog (Unity Catalog / Purview) with client-level access boundaries.

9J — Final Batch: Mixed Domain Questions

Q244 [General] What is database normalization? Explain 1NF, 2NF, 3NF.

• 1NF (First Normal Form): each column contains atomic (indivisible) values. No repeating groups. Each row is unique.

• 2NF (Second Normal Form): 1NF + every non-key column is fully functionally dependent on the ENTIRE primary key (no partial dependency). Relevant for composite PKs.

• 3NF (Third Normal Form): 2NF + no transitive dependencies (non-key column depending on another non-key column).

Example violation: order_table(order_id, product_id, product_name). product_name depends on product_id (not on order_id). Move product_name to a products table.

Q245 [IBM] What is denormalization? When is it preferred over normalization?

Denormalization: intentionally introducing redundancy by combining tables. Reduces JOIN complexity at the cost of update anomalies and storage.

Prefer when: read-heavy workloads where JOINs are expensive, OLAP/DW (star schema dims are denormalized by design), caching frequently joined data, simplifying queries for BI tools.

Avoid in: OLTP with frequent updates (redundancy causes inconsistency), systems where storage is tightly constrained.

Q246 [Snowflake] What is query result caching in Snowflake? How long does it last?

Snowflake caches query results for 24 hours. An identical query (same SQL text, same user role, unchanged underlying data) returns from cache instantly at zero credit cost.

Cache is invalidated when: underlying table data changes, role changes, 24 hours expire.

Tip: result caching is why dashboards loading the same reports fast — first user pays compute, subsequent users get cache.

Q247 [Netflix] What is the N+1 query problem? How does it apply to data pipelines?

N+1 problem: instead of one query fetching all needed data, code runs 1 query to get N records, then 1 more query per record = N+1 total queries.

In pipelines: iterating over a Pandas DataFrame row-by-row and calling an API or DB per row. Catastrophic at scale.

```
# BAD: N+1 pattern
for row in df.iterrows():
    result = db.query(f"SELECT * FROM ref WHERE id = {row.id}")  # 1 query per row

# GOOD: batch lookup
ids = df["id"].tolist()
ref = db.query(f"SELECT * FROM ref WHERE id IN ({",".join(map(str,ids))})")
df = df.merge(ref, on="id")
```

Q248 [Amazon] How do you implement a dead letter queue (DLQ) pattern in a data pipeline?

DLQ: a separate queue/path where unprocessable messages are routed after N failed processing attempts. Prevents one bad record from blocking the entire pipeline.

```
# Kafka: configure DLQ in consumer
try:
    process(record)
except Exception as e:
    producer.send("topic-dlq", key=record.key, value=record.value)
    log.error(f"Sent to DLQ: {e}")
```

```
# S3 DLQ pattern in PySpark
good = df.filter(F.col("_corrupt").isNull())
bad  = df.filter(F.col("_corrupt").isNotNull())
bad.write.mode("append").json("s3://pipeline/dlq/2024-01-15/")
```

DLQ records should be reviewed and reprocessed (with fixes) or discarded with clear documentation.

Q249 [IBM] What is a shared-nothing architecture? How does Teradata implement it?

Shared-nothing: each node has its own private CPU, memory, and disk. No shared resources between nodes. Scales linearly by adding nodes.

Teradata: each AMP (Access Module Processor) is an independent processing unit with its own portion of data. No AMP accesses another AMP's disk directly. Data exchange only through the BYNET (Teradata's internal network).

Benefit: no resource contention between AMPs. Adding AMPs = linear scaling.

Q250 [Databricks] What is Auto Loader in Databricks? How does it differ from structured streaming with parquet source?

Auto Loader: Databricks-optimized incremental file ingestion. Uses file notification (S3 event notifications → SQS) or directory listing to detect new files. Handles schema inference and evolution automatically.

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format","json")
    .option("cloudFiles.schemaLocation","s3://schema/events")
    .load("s3://raw/events/"))
```

Advantages over plain streaming: scales to millions of files (directory listing becomes slow at scale), built-in schema inference + evolution, exactly-once file processing guarantee.

Q251 [Netflix] What is the difference between a hot, warm, and cold data tier?

• Hot: frequently accessed, low-latency (<10ms). Expensive. Redis, DynamoDB, in-memory caches. Real-time serving.

• Warm: accessed regularly but not in real time. Moderate latency (<1s). Snowflake/Redshift, S3 Standard + Delta. Active analytics.

• Cold: rarely accessed, high latency acceptable (seconds to minutes). Cheap. S3 Glacier, Azure Archive. Historical/compliance data.

Smart tiering: Auto Loader + S3 lifecycle policies move data through tiers based on access patterns automatically.

Q252 [S&P Global] What is Apache NiFi and when would you use it over Kafka?

NiFi: visual dataflow tool for routing, transforming, and mediating between systems. Strong at: file-based ingestion, protocol translation (FTP, SFTP, JDBC, HTTP → Kafka/S3), drag-and-drop ETL, data provenance tracking.

Use NiFi for: complex multi-protocol ingest (many source system types), regulated industries needing data provenance out-of-box, teams without heavy Kafka/Spark skills.

Use Kafka for: high-throughput event streaming, microservice decoupling, replay capability, large engineering teams.

Q253 [IBM] What is data serialization? Compare JSON vs Avro vs Protobuf.

• JSON: human-readable, verbose, schema not enforced, slow parse. Good for APIs, debugging.

• Avro: binary, compact, schema stored separately in registry. Fast parse. Schema evolution support. Standard for Kafka in enterprise.

• Protobuf: binary, very compact, schema in .proto files. Fastest serialization. Strong typing. Good for high-throughput gRPC microservices.

For Kafka data engineering: Avro + Schema Registry is the most common choice. 5-10x smaller than JSON, schema enforced, evolution controlled.

Q254 [Databricks] What is a logical vs physical data model?

Logical data model: technology-agnostic representation of data entities and their relationships. Business language. Entities, attributes, relationships, keys. No DB-specific types.

Physical data model: implementation-specific. Actual table/column names, data types, indexes, partitions, constraints for a specific database (Snowflake, Teradata, PostgreSQL). Derived from logical model.

In DW: logical model = star schema design. Physical model = actual CREATE TABLE DDL with Snowflake cluster keys, Teradata PI, or Redshift SORTKEY.

Q255 [Amazon] What is a graph database? When would a data engineer use one?

Graph DB: stores data as nodes (entities) and edges (relationships). Optimized for traversing relationships. Examples: Neo4j, Amazon Neptune, TigerGraph.

Data engineering use cases: fraud ring detection (traverse transaction networks), recommendation engines (user → liked → item → category), social network analysis, knowledge graphs.

When NOT to use: simple tabular data with few relationships — a relational DB or DW is more appropriate and familiar.

Q256 [Netflix] What is change data feed in Delta Lake?

Change Data Feed (CDF): Delta Lake feature that captures row-level changes (INSERT/UPDATE/DELETE) on a Delta table, similar to CDC for a data warehouse.

```
-- Enable CDF
ALTER TABLE events SET TBLPROPERTIES ('delta.enableChangeDataFeed'='true');

-- Read changes since version 5
changes = spark.read.format("delta")
    .option("readChangeFeed","true")
    .option("startingVersion",5)
    .table("events")
# Each row has _change_type: insert, update_preimage, update_postimage, delete
```

Use CDF to propagate incremental changes to downstream Gold tables without full reprocessing.

Q257 [IBM] What are the phases of a data engineering project?

1. Discovery: understand business requirements, identify data sources, assess data quality.
2. Data profiling: explore source data — volume, distribution, nulls, uniqueness, formats.
3. Architecture design: choose tools, define data flow, design schemas.
4. Development: build ingestion, transformation, and serving layers. Write tests.
5. Validation: data quality checks, reconciliation against source, stakeholder sign-off.
6. Production deployment: CI/CD, monitoring, alerting, runbooks.
7. Ongoing operations: SLA monitoring, incident response, schema evolution, performance tuning.

Q258 [Teradata] What is workload management in Teradata TASM?

TASM (Teradata Active System Management): Teradata's workload management framework.

- Workload definitions: classify queries by user/app/query band into workloads.
- Priority: assign CPU/AMP allocation priorities per workload.
- Throttles: limit concurrent queries per workload to prevent runaway queries.
- Planned environments: different resource allocations for business hours vs batch.

Equivalent concepts: Snowflake virtual warehouses, Databricks cluster pools, Redshift workload management queues.

Q259 [Amazon] What is a Data Product? How does it differ from a dataset?

Dataset: a collection of data in a table or file. No ownership contract, no SLA, no quality guarantee.

Data Product: a dataset + ownership + documentation + quality SLA + discoverability + versioning + access control. Treated like a software product with consumers.

Data product principles: discoverable (in catalog), addressable (stable identifier), trustworthy (DQ checks + SLA), self-describing (schema + docs), interoperable (standard format), valuable (serves real business need).

Q260 [General] What monitoring metrics do you track for a production data pipeline?

- Pipeline SLA: did it complete within agreed time? (Airflow task duration vs SLA)
- Data freshness: MAX(updated_at) vs now.
- Row counts: today vs 7-day average. Alert on >20% deviation.
- Error rates: failed tasks, dead letter queue depth.
- Data quality pass rate: % of DQ checks passing.
- Kafka consumer lag: messages behind for streaming pipelines.
- Cluster resource utilization: CPU, memory, disk for Spark jobs.
- Cost: cloud spend per pipeline run (Databricks DBU, Snowflake credits, S3 data scanned).

Q261 [Databricks] How do you handle schema changes in a production dbt pipeline without breaking downstream?

• Add new columns: safe. Existing consumers are unaffected. Document in schema.yml.

• Rename column: breaking change. Use a transition period with both old and new column names (old column as alias), notify consumers, then deprecate old name.

• Remove column: breaking change. Never remove without coordination. Add deprecation notice in docs first.

• Change type: may be safe (INT→BIGINT) or breaking (STRING→INT). Test in staging before prod.

• dbt contracts: enforce schema with contracts so accidental breaking changes fail the dbt run before they reach production.

Q262 [Netflix] What is the difference between a MERGE and an UPSERT?

They mean the same thing functionally: insert if not exists, update if exists.

• MERGE: standard SQL:2003 syntax with WHEN MATCHED / WHEN NOT MATCHED clauses. Supports INSERT + UPDATE + DELETE in one statement.

• UPSERT: informal term. Some DBs implement as INSERT ... ON CONFLICT (PostgreSQL), INSERT OR REPLACE (SQLite), REPLACE INTO (MySQL).

```
-- Snowflake / Spark MERGE:
MERGE INTO target USING source ON target.id = source.id
WHEN MATCHED THEN UPDATE SET target.val = source.val
WHEN NOT MATCHED THEN INSERT (id, val) VALUES (source.id, source.val);
```

Q263 [IBM] What is a lookup join vs a regular join?

A lookup join (broadcast join in Spark, or small table join) is a JOIN where one side is small enough to cache in memory on all workers — no shuffle needed.

In Flink: Lookup Join queries an external system (Cassandra, DynamoDB, JDBC) for each streaming record. Different from a regular join which requires both sides as streams or static tables.

```
-- Flink lookup join (enrich stream with DB lookup)
SELECT s.user_id, d.country FROM stream_events s
JOIN dim_users FOR SYSTEM_TIME AS OF s.proc_time AS d
ON s.user_id = d.user_id;
```

Q264 [Amazon] What is a star join query? Why is it efficient in columnar stores?

Star join: a fact table joined to multiple dimension tables simultaneously. Pattern of a star schema query.

```
SELECT d.product_name, t.month, SUM(f.revenue)
FROM fact_sales f
JOIN dim_product d ON f.product_key = d.product_key
JOIN dim_date t ON f.date_key = t.date_key
GROUP BY 1,2;
```

Efficient in columnar: reads only needed columns (revenue from fact, product_name from dim_product, month from dim_date). Column store skips all other columns. Bitmap indexes on dim keys enable fast filtering.

Q265 [Databricks] What is query federation? Give an example.

Query federation: querying data across multiple separate data stores in a single SQL statement without moving data.

```
-- Snowflake: query S3 external table + Snowflake table together
SELECT s.user_id, e.event_type
FROM snowflake_table s
JOIN external_stage_table e ON s.id = e.id;
```

-- Other examples: Presto/Trino federates across S3, Hive, PostgreSQL, Kafka in one query.

-- AWS Athena federates across S3, DynamoDB, RDS via Lambda connectors.

Useful for: avoiding data movement for ad-hoc cross-system analysis.

Q266 [General] What is a WAL (Write-Ahead Log) and how does it enable CDC?

WAL: every change to a database is first written to a sequential log before being applied to data files. Ensures durability and crash recovery.

CDC via WAL: Debezium reads the PostgreSQL WAL (or MySQL binlog, MongoDB oplog) and streams every INSERT/UPDATE/DELETE as an event to Kafka — without any impact on source query performance.

This is why log-based CDC is preferred: zero query overhead, captures all changes including DELETEs, ordered stream.

Q267 [IBM] What is the difference between optimistic and pessimistic concurrency control?

Pessimistic: assume conflicts are likely. Lock the resource before reading/writing. Other transactions wait. Guarantees no conflicts but reduces throughput.

Optimistic: assume conflicts are rare. Read and write without locking. At commit time, check if data changed since you read it — if yes, abort and retry.

Delta Lake uses optimistic concurrency control: writes succeed independently; at commit, transaction log is checked for conflicts. If conflict exists, retry. If no conflict, commit succeeds.

Q268 [Netflix] What is an API rate limit and how do you handle it in a data ingestion pipeline?

Rate limit: external API restricts N requests per second/minute/day.

Handle in pipeline:

- Exponential backoff + jitter: on 429 (rate limit) response, wait 2^retry_count + random seconds.
- Request queuing: batch requests, send at controlled rate.
- Caching: cache API responses to avoid re-fetching same data.
- Pagination handling: respect cursor/page tokens, process pages sequentially or slowly.

```
import time
def fetch_with_retry(url, max_retries=5):
    for i in range(max_retries):
        resp = requests.get(url)
        if resp.status_code == 429:
            time.sleep(2**i + random.random())
        else: return resp.json()
```

Q269 [Databricks] What is MLflow and how do data engineers interact with it?

MLflow: open-source ML lifecycle platform. Tracks experiments, packages models, manages deployments.

Data engineer interactions:

- Log feature datasets used for training (data versioning for reproducibility).
- Register feature pipelines that produce training datasets.
- Deploy ML models as Spark UDFs for batch scoring in pipelines.
- Track model inference pipeline performance (latency, throughput, drift).

```
import mlflow
with mlflow.start_run():
    mlflow.log_param("num_features", 50)
    mlflow.log_metric("rmse", 0.12)
    mlflow.spark.log_model(model, "spark-model")
```

Q270 [Amazon] What is the difference between ETL testing and production data validation?

ETL testing (pre-production): verify transformation logic is correct on sample data. Unit tests on transformation functions. Integration tests on full pipeline with test fixtures.

Production data validation (post-load): verify data in production matches expectations. Row counts match source, aggregates reconcile, no new NULLs, no unexpected duplicates.

Both are needed. ETL tests catch logic bugs before production. Production validation catches issues with real data that tests couldn't anticipate (unexpected source changes, volume spikes).

Q271 [General] What is idempotency in REST APIs and how does it apply to data pipeline design?

REST idempotency: calling the same endpoint N times produces the same result as calling it once. GET, PUT, DELETE are idempotent. POST is not (creates new resource each time).

Data pipeline analogy:

- Idempotent write = PUT: running the pipeline for date=2024-01-15 twice produces exactly the same rows for that date. Use overwrite partition or MERGE.
- Non-idempotent write = POST: append mode. Running twice doubles the rows. Breaks reconciliation.

Design principle: always design pipeline reruns to be safe. Treat reruns as the default, not the exception.

Q272 [IBM] What is columnar vs row-based query execution? How does vectorized execution improve it?

Row-based execution: process one row at a time through the operator tree. Each row passes through filter → project → aggregate individually. High per-row overhead.

Columnar execution: process a column at a time (batches of column values). Better CPU cache utilization, SIMD instruction use.

Vectorized execution (Apache Arrow, Snowflake, DuckDB): process batches of rows (vectors) at a time using CPU SIMD instructions. 10-100x faster than row-at-a-time for analytical queries. Spark 3+ uses vectorized Parquet reader.

Q273 [Netflix] How do you implement data contracts in practice?

A data contract specifies: schema (column names, types, nullability), business semantics, SLA (freshness, availability), quality guarantees, versioning policy.

```
# dbt: enforce schema contract
models:
  - name: fact_orders
    config:
      contract:
        enforced: true  # dbt fails if model columns don't match schema.yml exactly
    columns:
      - name: order_id
        data_type: bigint
        constraints: [{type: not_null}]
```

Store contracts in Git. Breaking changes require PR review. Consumers subscribe to change notifications.

Q274 [Databricks] What is Apache Hudi? When would you use it?

Hudi (Hadoop Upserts Deletes and Incrementals): open table format optimized for upsert-heavy workloads. Originated at Uber.

Two table types:

• Copy-on-Write (CoW): rewrites Parquet files on every upsert. Read-optimized. Good for read-heavy with occasional updates.

• Merge-on-Read (MoR): writes delta logs, compacts periodically. Write-optimized. Good for high-frequency upserts.

Use Hudi when: extremely high upsert rate (millions/sec), CDC use cases with frequent updates, Hive-centric ecosystems.

Otherwise: Delta Lake or Iceberg are more commonly chosen in 2025.

Q275 [S&P Global] What is a data pipeline SLA? How do you define and enforce it?

SLA (Service Level Agreement): contractual commitment on data availability and quality.

Define: "Gold layer customer metrics table available by 7:00 AM UTC daily with >99.5% monthly availability and <0.1% error rate."

Enforce in Airflow:

```
with DAG("daily_metrics", sla_miss_callback=notify_on_call,
         default_args={"sla": timedelta(hours=1)}) as dag:
    run_job = SparkSubmitOperator(...)
```

Monitor: track actual completion time vs SLA target. Report monthly SLA adherence percentage.

Consequence of miss: notify downstream consumers immediately, activate incident response, root cause within 24h.

9K — Final 30+ Questions to Complete 300+

Q276 [General] What is a window function frame and what are ROWS vs RANGE?

Frame: subset of the window partition considered for each row's calculation.

• ROWS BETWEEN: physical row offsets. ROWS BETWEEN 6 PRECEDING AND CURRENT ROW = exactly 7 rows regardless of values.

• RANGE BETWEEN: logical value range. RANGE BETWEEN INTERVAL '6' DAY PRECEDING AND CURRENT ROW = all rows within 6 days by value.

Use ROWS for rolling N-row windows (safer). Use RANGE for value-range windows where you want all rows within a value gap included.

Q277 [Netflix] What is the difference between CUBE and ROLLUP in terms of output rows?

For N dimensions:

• ROLLUP produces N+1 grouping combinations (hierarchical: all, drop last, drop last two... grand total).

• CUBE produces 2^N combinations (every possible subset of dimensions).

ROLLUP(A,B,C) → (A,B,C), (A,B), (A), () = 4 groupings.

CUBE(A,B,C) → (A,B,C),(A,B),(A,C),(B,C),(A),(B),(C),() = 8 groupings.

Q278 [IBM] How do you prevent SQL injection in data pipelines?

• Never concatenate user input into SQL strings.

• Use parameterized queries / bind variables:

```
# BAD:
query = f"SELECT * FROM users WHERE id = {user_id}"

# GOOD: parameterized
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

• Use ORM or query builder libraries that auto-parameterize.

• Validate and whitelist inputs before using in dynamic SQL.

• Apply principle of least privilege: pipeline service accounts have only SELECT on needed tables.

Q279 [Databricks] What is Z-ordering and how does it differ from partitioning?

Partitioning: physically separates data into directories by column value (e.g., event_date=2024-01-15/). Great for high-cardinality date filters. But too many small partitions = small file problem.

Z-ordering: co-locates related data WITHIN files using a space-filling curve across multiple columns. Does not create directory structure. Good for multi-column filters on high-cardinality columns.

```
OPTIMIZE events ZORDER BY (user_id, content_id);
```

Combine both: partition by date (coarse), Z-order by user_id + content_id (fine-grained within each date partition).

Q280 [Amazon] What is connection pooling in databases? Why does it matter for data pipelines?

Connection pooling: maintain a pool of pre-opened database connections reused across requests. Avoids the overhead of opening/closing a DB connection per query.

In pipelines: Airflow operators, dbt, and custom scripts all open DB connections. Without pooling, each task opens a fresh connection (slow, can exhaust DB connection limits).

```
# SQLAlchemy connection pool
engine = create_engine(url, pool_size=10, max_overflow=20, pool_pre_ping=True)
```

pool_pre_ping: checks if connection is alive before using it (avoids stale connection errors after idle period).

Q281 [Teradata] What is AMP (Access Module Processor) skew and how do you fix it?

AMP skew: one AMP has significantly more data than others. Causes: NUPI with hot key value, PI chosen incorrectly.

```
-- Detect skew
SELECT hashamp(hashbucket(hashrow(customer_id))) amp_num, COUNT(*) rows
FROM orders GROUP BY 1 ORDER BY 2 DESC;
```

Fix:

1. Change PI to a more evenly distributed column.
2. Use NUPI + partition to spread data.
3. For JOIN skew: use two-step aggregation or hash distribution tricks.

Q282 [Snowflake] What is a Snowflake Stage and how is it used for data loading?

Stage: a named storage location (S3, Azure Blob, GCS, or Snowflake internal) where data files are staged before COPY INTO.

```
-- Create external stage pointing to S3
CREATE STAGE my_s3_stage
  URL = 's3://my-bucket/data/'
  CREDENTIALS = (AWS_KEY_ID='...'  AWS_SECRET_KEY='...');

-- List files in stage
LIST @my_s3_stage;

-- Load from stage
COPY INTO orders FROM @my_s3_stage/orders/2024-01-15/
  FILE_FORMAT = (TYPE = PARQUET);
```

Q283 [General] What is a message broker? Compare Kafka vs RabbitMQ vs SQS.

• Kafka: distributed log. Persistent, replayable, high-throughput. Pull-based. Consumers track their own offsets. Best for event streaming, CDC, data pipelines.

• RabbitMQ: traditional message queue. Push-based. Messages deleted after consumption. AMQP protocol. Best for task queues, microservice async communication.

• AWS SQS: managed queue service. At-least-once delivery. Auto-scaling. Best for AWS-native serverless event processing (Lambda triggers).

For data engineering pipelines at scale: Kafka wins on throughput, replayability, and ecosystem (Kafka Connect, ksqlDB, Schema Registry).

Q284 [IBM] What is a hash collision and how does it affect distributed systems?

Hash collision: two different keys produce the same hash value. In a hash table, this is handled by chaining or open addressing.

In distributed data systems: if two keys hash to the same partition, they both land on the same node — contributing to skew if one key is very hot.

Practical impact: in Spark hash join, very high collision rates in the build-side hash table slow down probe phase. In Kafka, poor partition key choice causes uneven partition distribution.

Mitigation: use high-quality hash functions (MurmurHash3 in Spark), salt keys to reduce collision probability.

Q285 [Netflix] What is the difference between TRUNCATE and DELETE in terms of transactions and locking?

• DELETE: DML. Row-by-row. Logged. Acquires row-level locks. Can be rolled back within a transaction. Slow on large tables.

• TRUNCATE: DDL in most DBs. Deallocates data pages. Minimal logging (only page deallocation logged). Table-level lock. Cannot be rolled back in MySQL (auto-commit). In PostgreSQL: TRUNCATE IS transactional.

For clearing a staging table before reload: TRUNCATE is preferred (100x faster on large tables).

Q286 [Amazon] What is a data pipeline orchestrator? Compare Airflow vs Prefect vs Dagster.

• Airflow: most widely deployed. Python DAGs. Rich ecosystem. But: DAGs defined statically, testing locally is complex, scheduler can be a bottleneck at scale.

• Prefect: more Python-native. Dynamic task generation. Better local testing. Cloud-hosted option (Prefect Cloud). Easier learning curve.

• Dagster: asset-centric (model pipelines as producing data assets, not running tasks). Strong software engineering practices (testing, types). Best for teams wanting DX + observability.

Industry default: Airflow (especially managed MWAA or Astronomer). Dagster gaining fast in modern data teams.

Q287 [Databricks] How do you implement logging in a PySpark job?

```
import logging

# Configure logger
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

log.info(f"Processing date: {run_date}")
log.info(f"Input records: {df.count()}")

try:
    result = transform(df)
    log.info(f"Output records: {result.count()}")
except Exception as e:
    log.error(f"Job failed: {e}", exc_info=True)
    raise
```

In Databricks: logs go to cluster driver logs. Send to external log aggregator (Splunk, Datadog, CloudWatch) via Log4j configuration.

Q288 [S&P Global] What is parallel vs serial pipeline execution? When is parallelism limited?

Parallel: independent tasks run simultaneously. Reduces total wall-clock time. Airflow: tasks without dependencies run in parallel (up to concurrency limits).

Serial: tasks run sequentially. Required when: task B needs output of task A, resource contention, or order-dependent side effects.

Parallelism limits: shared resource bottlenecks (DB connection limit, API rate limit), upstream task dependency chains, Airflow worker pool size, cluster autoscaling lag.

Optimize: identify the critical path (longest sequential chain), parallelize independent tasks, use Airflow pools to throttle resource-constrained tasks.

Q289 [Netflix] What is a soft delete vs hard delete in a data warehouse context?

Hard delete: remove the row. Data gone. Breaks SCD Type 2 history.

Soft delete: mark row as deleted with a flag (is_deleted=TRUE) or a deletion timestamp. Row stays in table. History preserved.

```
-- Soft delete pattern
UPDATE dim_customer SET is_deleted=TRUE, deleted_at=CURRENT_TIMESTAMP WHERE
customer_id=123;

-- Queries exclude deleted rows
SELECT * FROM dim_customer WHERE is_deleted IS FALSE OR is_deleted IS NULL;
```

In DW: always prefer soft delete for audit trail. Hard delete only for GDPR right-to-erasure requests.

Q290 [IBM] What is data masking at the pipeline level vs at the serving layer?

Pipeline-level masking: PII is transformed/hashed BEFORE writing to storage. Downstream always sees masked data. Simple, irreversible for analyst consumers.

Serving-layer masking: PII stored in full, dynamically masked at query time based on user role (Snowflake Dynamic Data Masking, Lake Formation column-level security). Reversible for privileged users.

Best practice: pipeline-level pseudonymization + serving-layer dynamic masking for different tiers of access.

Q291 [Amazon] Explain the write-ahead log (WAL) pattern and its use in distributed DBs.

WAL: before applying any change to data pages, write the change to a sequential log first. If DB crashes, log is replayed to recover.

Ensures durability: D in ACID. Even if crash happens mid-write, the log has the complete intention of the transaction.

Distributed use: WAL is the foundation of CDC (Debezium reads it), replication (leader streams WAL to followers), point-in-time recovery.

Q292 [Databricks] How do you handle timezone issues in data pipelines?

- Store all timestamps in UTC. Never store local time in the database.
- Convert to local time only at the presentation/reporting layer.

```
# PySpark: ensure UTC storage
spark.conf.set("spark.sql.session.timeZone","UTC")
df.withColumn("ts_utc", F.to_utc_timestamp(F.col("ts_local"), "America/New_York"))

# When reading CSV with no TZ info, specify the TZ
df = spark.read.option("timestampFormat","yyyy-MM-dd HH:mm:ss").csv(...)
```

Beware: daylight saving time transitions create ambiguous or skipped local times. UTC avoids all this.

Q293 [Netflix] What is a canary deployment for a data pipeline?

Canary: roll out new pipeline version to a small subset of data/users first. Monitor for errors. Gradually increase.

Data pipeline canary: run new pipeline on 1% of data (e.g., one partition or one region). Compare output to old pipeline. If metrics match and no errors, expand rollout.

Also applies to schema changes: migrate one table at a time, validate downstream, then migrate rest.

Q294 [IBM] What are the differences between structured, semi-structured, and unstructured data?

- Structured: fixed schema, tabular. SQL databases, CSV. Easy to query. Example: orders table.
- Semi-structured: flexible schema, self-describing. JSON, XML, Avro, Parquet with nested fields. Example: API responses, clickstream events.
- Unstructured: no predefined schema. Images, audio, video, PDFs, raw text. Requires ML/NLP to extract structure.

Modern data lakes handle all three. Parquet with nested types bridges structured + semi-structured. Data lakehouses query semi-structured natively (Snowflake VARIANT, Delta with schema inference).

Q295 [General] What is Apache Spark vs Apache Hadoop MapReduce? Why did Spark win?

MapReduce: 2-phase (map + reduce) paradigm. After each step, results written to HDFS (disk). Multi-step jobs require many disk reads/writes.

Spark: in-memory processing. Chains transformations in memory with lazy evaluation. Only materializes to disk when necessary (shuffle). Result: 10-100x faster than MapReduce for iterative algorithms.

Also: Spark provides a unified API (batch + streaming + ML + SQL). MapReduce required different frameworks for different tasks.

Spark won by: speed (in-memory), ease of use (Python/Scala API), versatility (one framework for all workloads).

Q296 [Databricks] What is Unity Catalog and why is it important for enterprise data governance?

Unity Catalog: Databricks centralized governance layer across all workspaces.

- Three-level namespace: catalog.schema.table
- Fine-grained access: GRANT SELECT ON TABLE to roles
- Column-level security: masking policies on PII columns
- Row-level security: row access filters
- Data lineage: automated column-level lineage across all notebooks, jobs, pipelines
- Audit logs: full access audit trail
- Delta Sharing: securely share data across clouds/orgs without copying

Critical for: GDPR compliance, cost attribution, data discovery, security audits.

Q297 [Netflix] How do you efficiently compute a top-K query on a very large dataset?

```
-- SQL: use LIMIT (most optimizers avoid full sort)
SELECT user_id, watch_minutes FROM watch_events
ORDER BY watch_minutes DESC LIMIT 100;

# PySpark: avoid sorting entire dataset
df.orderBy(F.desc("watch_minutes")).limit(100)  # Catalyst optimizes to partial sort

# For top-K per group: use ROW_NUMBER window (partition eliminates full sort)
df.withColumn("rn",
F.row_number().over(Window.partitionBy("category").orderBy(F.desc("score"))))
  .filter(F.col("rn") <= 10)
```

At extreme scale (trillions of rows): use approximate top-K algorithms (Count-Min Sketch + heap) for O(N) time.

Q298 [Amazon] What is a slowly changing attribute? How do you model employee salary history?

Salary changes over time — a slowly changing attribute. Model with SCD Type 2:

```
CREATE TABLE dim_employee_salary (
  salary_key       BIGINT GENERATED ALWAYS AS IDENTITY,
  employee_id      INT,
  salary           DECIMAL(12,2),
  effective_start  DATE,
  effective_end    DATE DEFAULT '9999-12-31',
  is_current       BOOLEAN DEFAULT TRUE
);
```

Query: "What was employee 123's salary on 2023-06-01?"

```
SELECT salary FROM dim_employee_salary
WHERE employee_id=123 AND '2023-06-01' BETWEEN effective_start AND effective_end;
```

Q299 [General] What is a data pipeline vs a data workflow vs a data process?

- Data process: a single transformation step (a Python script, a SQL query, a Spark job).
- Data pipeline: a sequence of connected processes that moves and transforms data from source to destination.
- Data workflow: an orchestrated set of pipelines with dependencies, scheduling, and monitoring (an Airflow DAG containing multiple pipeline tasks).

Hierarchy: Process < Pipeline < Workflow.

Q300 [IBM] What is EXPLAIN in SQL? What plan operators do you look for?

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 123 AND order_date > '2024-01-01';
```

Key plan operators to look for:

• Seq Scan: full table scan. Add index if table is large and filter is selective.
• Index Scan / Index Only Scan: good. Using an index.
• Hash Join: standard for large-large joins.
• Nested Loop: good only if outer side is tiny.
• Sort: expensive if on very large dataset without index.
• Hash Aggregate: expected for GROUP BY.
• Estimated vs actual rows: large discrepancy → stale statistics.

Q301 [Snowflake] What is a dynamic table in Snowflake?

Dynamic Table: Snowflake-managed table that automatically refreshes based on its SQL definition when upstream data changes. Similar to a materialized view but with more control.

```
CREATE OR REPLACE DYNAMIC TABLE customer_metrics
  TARGET_LAG = '1 hour'  -- refresh within 1 hour of upstream change
  WAREHOUSE = my_wh
AS
SELECT customer_id, SUM(amount) total_spend FROM orders GROUP BY 1;
```

Advantage over scheduled dbt: Snowflake manages refresh timing automatically. No Airflow DAG needed for simple refresh pipelines.

Q302 [Databricks] What is the difference between saveAsTable and write.parquet in Spark?

```
df.write.parquet("s3://bucket/path/")  -- writes files only, no catalog entry
df.write.saveAsTable("db.table_name")  -- writes files + registers in Hive
Metastore/Unity Catalog
```

saveAsTable creates a catalog entry (metadata: location, schema, format) — queryable by name in SQL.

write.parquet creates files only — must specify path in every query. No discoverability in catalog.

For production tables: always use saveAsTable or equivalent (CREATE TABLE ... LOCATION ...) for discoverability and governance.

Q303 [General] What is a surrogate key vs a natural key in dimension tables?

Natural key: identifier from the source system (customer_email, product_SKU). Meaningful but may change, not unique across source systems.

Surrogate key: synthetic integer generated by the DW (customer_key IDENTITY). Meaningless but stable, unique, compact (INT vs STRING), immutable.

Best practice: always use surrogate keys in fact-dim relationships. Store natural keys in dims as attributes (for joining back to source). Surrogate key insulates DW from source system changes.

Q304 [Amazon] How do you secure a data lake on AWS? List the security layers.

1. S3 bucket policies: restrict access to specific IAM roles/accounts.
2. AWS Lake Formation: column/row-level access control on top of Glue catalog.
3. IAM roles: principle of least privilege — pipeline roles get only what they need.
4. Encryption at rest: S3 SSE-KMS for all data. Customer-managed keys.
5. Encryption in transit: TLS on all connections (Spark, JDBC, API calls).
6. VPC endpoints: keep S3 traffic within AWS network (no public internet).
7. CloudTrail: audit log of all S3 and Glue API calls.
8. Macie: auto-detect PII in S3 buckets.

Q305 [Netflix] What is a self-service data platform? What are its key capabilities?

Self-service: enables analysts, data scientists, and business users to access, explore, and use data without needing data engineering for every request.

Key capabilities:

• Data catalog: searchable inventory of all available datasets.
• Access request workflow: automated provisioning via request portal.
• Query interface: SQL editor (Snowflake UI, Mode, Databricks SQL).
• Pre-built data marts: curated Gold layer tables ready for analysis.
• Data documentation: definitions, owners, SLAs for each dataset.

• Usage analytics: track who is using what data for governance and prioritization.

CHAPTER 10 — CHEAT SHEETS & REFERENCE

Window Function Syntax Reference

```
function() OVER (
  [PARTITION BY col1, col2]
  [ORDER BY col3 ASC|DESC]
  [ROWS|RANGE BETWEEN frame_start AND frame_end]
)

-- Frame options:
UNBOUNDED PRECEDING AND CURRENT ROW   -- running total (default with ORDER BY)
UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  -- whole partition
6 PRECEDING AND CURRENT ROW           -- 7-row rolling window

-- Ranking:     ROW_NUMBER, RANK, DENSE_RANK, NTILE(n), PERCENT_RANK, CUME_DIST
-- Navigation:  LAG(col,n,default), LEAD(col,n,default), FIRST_VALUE, LAST_VALUE,
NTH_VALUE
-- Aggregate:   SUM, AVG, MIN, MAX, COUNT  (all work as window functions)
```

PySpark One-Liner Reference

```
from pyspark.sql import functions as F, Window

# Read/Write
spark.read.format("delta|parquet|json|csv").option("mergeSchema","true").load("path")
df.write.format("delta").mode("overwrite|append").partitionBy("col").save("path")

# Filter / Select / Transform
df.filter(F.col("x")>10).select("a","b",F.col("c").alias("d"))
df.withColumn("new", F.coalesce("a","b")).dropDuplicates(["id"]).fillna({"col":0})

# Aggregation
df.groupBy("k").agg(F.sum("v"),F.count("*"),F.avg("v"),F.countDistinct("id"))

# Window
w = Window.partitionBy("k").orderBy("ts")
df.withColumn("rn", F.row_number().over(w))
df.withColumn("lag1", F.lag("v",1).over(w))
df.withColumn("rtot", F.sum("v").over(w.rowsBetween(Window.unboundedPreceding,0)))

# Joins
df1.join(F.broadcast(df2),"key","left")
df1.join(df2,"key","left_anti")   # anti-join

# JSON / Struct
df.withColumn("p", F.from_json(F.col("raw"),schema)).select("p.*")
df.withColumn("tag", F.explode("tags"))  # array explode

# Streaming
stream.withWatermark("event_time","1 hour")
      .groupBy(F.window("event_time","5 minutes"),"user_id")
      .agg(F.count("*")).writeStream.format("delta")
      .option("checkpointLocation","s3://ckpt/").start("s3://out/")
```

Common Interview Traps — Quick Reference

Trap	How to Avoid

NOT IN + NULL	If subquery returns any NULL, NOT IN returns empty. Use LEFT JOIN anti-join.
LAST_VALUE()	Default frame RANGE...CURRENT ROW makes it behave like CURRENT ROW. Add explicit ROWS BETWEEN UNBOUNDED...UNBOUNDED.
collect() OOM	Never collect() a large DF. Use write() or show(n).
UDF slowness	Python UDFs: serialize/deserialize penalty. Use native F. functions or Pandas UDF.
coalesce() limitation	coalesce() can only reduce partitions. repartition() for increase/rebalance.
VACUUM < 7 days	Breaks concurrent readers. Never set retention below 168 hours in production.
Function on partition col	WHERE YEAR(event_date)=2024 breaks partition pruning. Use date range instead.
Fan-out JOIN	Non-unique join key multiplies rows. Always COUNT before/after joining.
HAVING vs WHERE	WHERE filters rows before aggregation. HAVING filters after. Mistake = wrong results.
AQE not enabled	Default Spark 3: AQE enabled; but spark.sql.shuffle.partitions default=200 may still be wrong.
SET table in Teradata	SET table enforces uniqueness on every insert. Use MULTISET for bulk loads.
PI skew in Teradata	NUPI with high-frequency value → one AMP overloaded. Check AMP distribution before choose PI.
SCD2 forgetting is_current	Querying SCD2 without is_current=TRUE or date filter returns all historical versions.
Schema drift	New column in source silently breaks downstream CAST or SELECT *. Add schema validation at ingest.
Event time vs processing time	Streaming: always window on event_time, not ingestion time. Otherwise late events skew windows.

Total Questions: 305 — You're Ready.

This booklet covers 305 real interview questions from Netflix, IBM, S&P Global, Teradata, Databricks, Snowflake, Stripe, Airbnb, Meta, and Amazon. SQL · PySpark · Kafka · Airflow · Delta Lake · dbt · System Design · Data Quality · Governance · Behavioral.

Good luck. You've prepared from every angle.